

www/

What is java and why java is pure object oriented?

Java is a pure object-oriented programming language, developed by Sun microsystems in 1995.

Java is a widely used programming language known for its platform independence. Platform independence means the programs written in java can run on any device that has a java Virtual Machine (JVM) i.e. known as write once run anywhere (WORA).

It is commonly used for developing mobile apps, web applications, and large systems software. Java is also known for its security features and robustness, making it a popular choice for enterprise-level applications.

Java is pure object-oriented language –

1 We cannot execute java program without class and class is a feature of OOP.

2 We cannot create programs without inheritance, and it is also a feature of OOP.

3 Java supports primitive data type then how it's pure object-oriented?

yes, java supports primitive data type, but java also supports wrapper classes, so we have to use wrapper classes instead of primitive data type in program, that means wrapper classes makes java purely object-oriented language.

what is diff between C++ and JAVA?

C++	JAVA
C++ is semi object-oriented	JAVA is pure object-oriented
C++ can use structure and union.	Java not use structure and union.
C++ use delete operator	Java not use delete operator use garbage collection.
C++ support external use of pointer.	Java use internal pointer but programmer not need to declare it manually.
C++ support operator overloading	Java does not support operator overloading.
C++ support friend function.	Java not support friend function.
C++ achieve abstraction using pure virtual function.	Java achieve abstraction using abstract keyword.
C++ not support to final keyword use const keyword for constant declaration.	Java use final keyword for declare constant, avoid function overriding and avoid inheritance.
C++ use template.	Java use generics for general programming
C++ provide data structure library using standard template library.	Java provide data structure implementation using collection framework.
C++ use inline function and macros.	Java does not support inline function and macros.

We can use call by value with object in C++.	In Java we must work with call by reference with object.
C++ support multiple inheritance using classes.	Java implements multiple inheritance using interface.
C++ support scope operator access global variable in local block.	Java not support global variable in java static work like as global.
C++ allow us to define function outside of class.	Java not allow define function outside of class must be define within class.
C++ not support this() constructor	Java support to this() constructor.
C++ use base class constructor concept for pass parameter to parent constructor from child class.	Java use super() constructor concept for pass parameter to parent constructor from child class.

Explain feature of java?

Pure Object-oriented: We cannot create java program without OOP features like as class, inheritance.

Platform independent: We can execute java program on any operating system without taking any third-party tool support, so it is platform independent.

Portable: The WORA (Write Once Run Anywhere) concept and platform-independent feature make Java portable. Now using the Java programming language, developers can yield the same result on any machine, by writing code only once. The reason behind this is JVM and bytecode. Suppose you wrote any code in Java, then that code is first converted to equivalent bytecode which is only readable by JVM. We have different versions of JVM for different platforms.

Robust and Secure: Robust means java handle the run time program termination means manage exception at run time as well as manage memory like as garbage collection at run time etc, so it is robust and secure means java provide some security API to us which help us to apply security on java application, so it is secure like as spring security etc.

Multithreaded: Java provide support to multithreading means using java we can achieve multi-threaded application which help us to improve performance of application as well as it helps us to manage servers, gaming API, multi-threaded application helps us to share resources etc.

Interpreted: Java use the compiler as well as interpreter under JVM which help us to interpreted byte code at run time and convert in machine code for execution purpose.

Huge API Library: Java provide huge inbuild classes to us for build application like as java provide collection framework which help us to implement data structure, java provide spring security API to us which help us to provide security to application etc.

What is byte code and why java develops byte code?

Byte code is an intermediate code, or it is machine understandable code means byte code is responsible to generate machine code with the JVM means byte code can accept by JVM and put in RAM and generate machine code from that and execute it and generate output.

The benefit of bytecode is it is platform independent code.

What is difference between byte code and machine code?

Aspect	Byte Code	Machine Code
Definition	Intermediate code between source code and machine code.	Set of instructions directly machine-understandable.
Processing	Processed by a virtual machine like Java Virtual Machine (JVM).	Processed by the Central Processing Unit (CPU).
Runnable	Non-runnable code until translated by an interpreter into machine code.	Directly runnable by the machine.
Format	Low-level code resulting from the compilation of a high-level language.	Set of instructions in binary (0s and 1s) format.
Platform	Compiled to run on JVM, making Java platform-independent.	Specific to the machine and its architecture.

What is diff between platform independency and cross platform?

Aspect	Platform Independency	Cross Platform
Definition	Refers to the ability of software to run on different platforms without modification.	Indicates the capability of software to run on multiple platforms with minimal modifications.
Scope	Focuses on the ability of a program to execute on various platforms without recompilation.	Encompasses the adaptability of a program to operate on different platforms with minor adjustments.
Modification Needed	Typically requires no or minimal changes to the code to run on different platforms.	May require slight adjustments or use of cross-platform frameworks to ensure smooth operation on diverse platforms.
Examples	Java programs are platform independent due to the use of bytecode and the Java Virtual Machine (JVM).	Cross-platform frameworks like Xamarin, React Native, and Flutter enable the development of applications that can run on multiple platforms.
Flexibility	Offers a high degree of flexibility in terms of deployment on various operating systems.	Provides a level of flexibility by leveraging cross-platform development tools and frameworks.

What is diff between semi object oriented and pure object oriented?

1. Semi Object oriented programming or Partial Object-Oriented programming

Semi object-oriented programming means not necessary to use object-oriented concept compulsory in code means we can use some feature of procedure oriented for develop application called as semi object oriented.

C++ is semi object-oriented language means using C++ we can achieve OOP concept as well as you can use all C feature in C++ i.e. can use Procedure oriented approach in C++ so we can it is called as semi object-oriented language.

2. Pure Object-Oriented Programming

Pure Object-Oriented means we cannot develop any application without using OOP feature called as pure object-oriented language.

like as java, c# are the pure object oriented means we cannot run any program of java or c# without class and class is feature of OOP, we cannot create any program java or c# without inheritance so inheritance is feature of OOP so we can say it is object oriented programming.

What is JVM and explain JVM architecture?

JVM stands for java virtual machine, and it is application present under JDK and JRE which is used for load class file means load byte code generated by java compiler and perform different operations on it means verify byte code, perform garbage collection, object memory allocation, thread management, memory management etc. these are the roles of JVM.

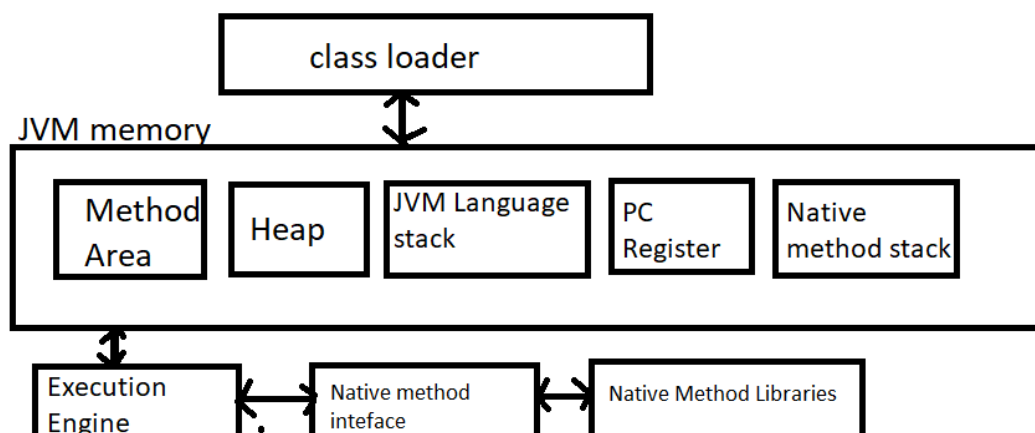
JVM responsible for managing run time environment required for java program.

JVM is an application which is responsible for accept byte code generated by compiler and put byte code in ram verify byte code and then convert in machine code and process on it and generate output.

JVM architecture -

JVM is responsible for manage the overall activity of Java program execution.

Means when we compile java program then java compiler creates .class file and store byte code in it and load .class file by JVM in memory and perform different operation on it means Allocate memory for class and its member, allocate memory for objects, destroy objects from memory as well as manage n number of things for java program shown in following diagram or show by JVM architecture.



JVM Architecture: It contains classloader, memory area, execution engine etc.

1 Classloader:

In classloader main three activities performed: Loading, Linking, Initialization

Loading – This class loader reads the “.class” file, generate corresponding data and save it in the method area.

Linking – It perform (1)verification means it ensures the correctness of the .class file i.e. it checks whether this file is properly formatted and generated by a valid compiler or not. If not then gives java.lang.VerifyError. then (2)Preparation means JVM allocates memory for class static variables and initializing the memory to default values. then (3)Resolution means it is the process of replacing symbolic references from the type with direct references.

Initialization – In this phase, all static variables are assigned with their values defined in the code and static block. There are three class loaders -

Bootstrap Classloader: it loads rt.jar files which contains all class files like java.lang package classes, java.util package classes.

Extension Classloader: It is child class of Bootstrap and parent of System classloader. it loads jar file from home/jre/lib/ext directory.

System Classloader: it loads classfiles from classpath. it known as application classloader.

2 Memory area allocated by JVM (JVM Memory):

1 Method area – In method class, all class level information likes class name, method and variables information etc. are stored, including static variables. There is only one method area per JVM, and it is shared resources.

2 Heap area – Information of all objects is stored in heap area. There is also one heap area per JVM. It is also a shared resource.

3 Stack area – For every thread, JVM creates one run-time stack which is stored here. It holds local variables and partial results and plays a part in method invocation and return. After thread terminates, its run-time stack will be destroyed by JVM. It is not a shared resource.

4 Program Counter Register (PC Register) – Store address of current execution instruction of a thread. Each thread has separate PC Registers.

5 Native Method Stack – For every thread, a separate native stack is created. It contains all native methods information used in the application.

3 Execution Engine: It executes the “.class” (bytecode). It classified into three parts: Interpreter, Just-In-Time (JIT) Compiler, Garbage Collector. Interpreter interprets the bytecode line by line and then executes. JIT used to increase the efficiency of an interpreter. It compiles the entire byte code. Garbage Collector destroys un-referenced objects.

4 Java Native Interface: It is an interface that interact with the Native Method Libraries and provides the native libraries(C, C++) required for the execution. It enables JVM to call C/C++ libraries and to be called by C/C++ libraries which may be specific to hardware.

What is diff between JDK JRE and JVM?

Parameter	JDK	JRE	JVM
Full Form	Java Development Kit	Java Runtime Environment	Java Virtual Machine
Implementation	JDK = Development Tools + JRE	JRE = Libraries for running applications + JVM	JVM = Runtime environment for executing Java bytecode
Purpose	Used for developing and executing Java programs	Used for running Java programs	Provides a runtime environment for executing Java bytecode
Platform Dependency	Yes, needs to be installed separately for each operating system	Yes, needs to be installed separately for each operating system	No, available for different hardware and software platforms
Tools Included	Yes, includes development tools like compilers and debuggers	No, does not include development tools	No, does not include development tools
Usage	Used by developers to write, compile, and debug Java code	Used by end-users to run Java applications	Used by JVM to execute Java bytecode
Relationship	Includes JRE	Included in JDK	N/A, JVM is a component of both JDK and JRE

What is platform independency?

Here platform is operating system and byte code is platform independent means it is operating system independent means byte code can execute on any operating system so java is platform independent language.

Means if you generate byte code on windows operating system it will execute on Linux, Unix, windows etc.

What is diff between platform independency and cross platform?

Platform Independent software : Platform independent software in its strict sense means –

- Install anywhere and run everywhere – it doesn't matter where you are. Although you may get different installers (for different platforms), this would be just for users ease. There would be options for you to install directly from the source code.

A Multi-platform or cross-platform software : software is available on more than one platforms (operating systems). This could mean two things -

1. The software is provided with different builds / applications for different platforms [different packages for windows, this one for linux].
2. The software can be run (with the same download) on multiple platforms but NOT all.

Why main method is static?

Because java main function present in class and we run java program using class which contain main method and main method call by JVM and if method present in class then need to create object but when we create object then constructor get executed means if main method not static then JVM need to create object of class and if user define parameterized constructor in class where main method is exist then at the time object creation JVM need to pass parameter but JVM cannot pass parameter to user constructor so avoiding this possibility JVM want to call main method without object so it is static because static member can call without object.

Can we overload main method?

yes we can overload the main method because if we have main method with different data type except string so it is overloaded by JVM execute program using main method which contain string array as single parameter

Example:

```
class A
{
    public static void main(String x[])
    { System.out.println("I am original main"); }
    public static void main(int x)
    { System.out.println("it is user main"); }
}
```

JVM call this method because JVM knows only one signature of main which contain String [] single parameter

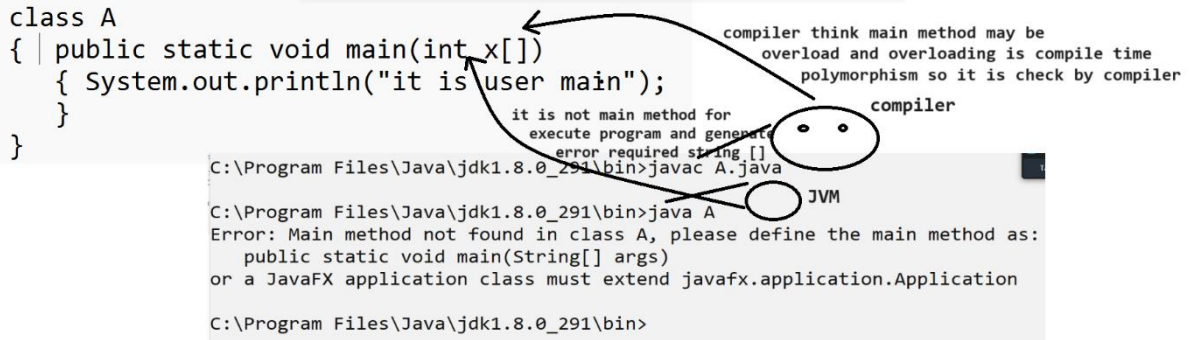
it is used main define main method

Note: finally we have two main method with different parameter as per method overloading rule it is main method overloading in program.

Can we pass different parameter in main method other than string and what happen?

If we pass different parameter in main except string then program get compile but not run because

Because when we compile program then compiler think main method may be overloaded but when we run program then JVM knows only one method which contain string array as argument and if JVM not found this parameter in main method then generate error at run time



Can we execute java program without main?

Prior to JAVA 7, It was possible to execute a java program without a main method using a static initialization block. The java Virtual Machine (JVM) would first load the class, then execute all static blocks, and finally look for the main method and use it.

Example:

```
class StaticInitializationBlock {
    static {
        System.out.println("class without a main method");
    }
}
```

In the above example, the static block is used to execute the program without a main method. However, it is important to note that from JAVA 7 onwards, the main method is mandatory, and the JVM checks for its presence before initializing the class.

If a java program doesn't contain the main method, an error will be thrown, indicating that the main method was not found in the class.

Is there possible to execute code before main function?

In java, it is not possible to execute code before the main function. The main function method serves as the entry point for a java program, and the Java Virtual Machine (JVM) looks for this method to start the execution of the program.

Prior to JAVA 7, it was possible to execute a java program without a main method using static initialization blocks. However, from JAVA 7 onwards the main method is mandatory, and the JVM checks for its presence before initializing the class.

Example:

```
class StaticInitializationBlock {
    static {
        System.out.println("This code is executed before the main method");
    }
}
```



```
public static void main(String[] args) {  
    System.out.println("This is the main method");  
}  
}
```

It is important to note that this approach is no longer valid in modern versions of java, and the main method is now mandatory requirement from java 7 onwards.

What is command line argument and why use it?

Command line argument is parameter present in main function of String array and it is used for accept input at program run time and the first input of command line is at position of 0 to n-1.

Note: explain all content from command line argument topic with source code example.

What is Scanner class and why use Scanner class?

The Scanner class in Java is used to obtain user input and is found in the java.util package. It provides various methods to read and parse input of primitive types like int, double, etc. as well as strings. The Scanner class breaks the input into tokens using a delimiter, which is whitespace by default, and it can also use delimiters other than whitespace.

Use of Scanner Class :

To create an object of the Scanner class, the predefined object System.in, which represents the standard input stream, is usually passed. IT can also accept an object of the File class if input needs to be read from a file.

The Scanner class provides methods like nextInt(), nextDouble(), nextLine(), etc., to read different types of input.

It is widely used for parsing text for strings and primitive types using regular expressions, making it the simplest way to get input in JAVA.

The Scanner class is commonly used for tasks such as obtaining user input for console-based applications, parsing input from files, and controlling data format before entering it into a database. It provides a convenient way to interact with users and handle input in java programs.

What is array and why use array?

Array is a collection of similar type of data means we can store multiple values in single variable of same type.

What is diff between c array and java array?

1. C array allocate memory at compile time `int a[5]` and java array allocate memory at run time by using `new` keyword means in c language we can allocate memory of array at run time using `malloc` or `calloc` but it is mandatory to use every means we can allocate it without dynamic memory allocation function so we can say it is compile time
2. C array does not provide the length internally programmer need to calculate manually but java array provide length constant internally programmer does not need to calculate length manually
3. C array does not generate `ArrayIndexOutOfBoundsException` at run time expand size at run time, but java array generates `ArrayIndexOutOfBoundsException` if try to access values more than its index
4. Internally java array reference mark as object reference i.e. pointer but c array variable mark as primitive type of reference or pointer.

Example:

```
int a[];  
int []a;  
int a[] = new int[5] ;
```

What is anonymous array?

An anonymous array in java is an array without any name, created and used instantly for one-time usage. It can be passed as an argument of a method. The main purpose of an anonymous array is for instant use, and it is created without mentioning the size in square brackets. The number of values passing inside curly braces determines the size of the array. Anonymous array have not a reference so we cannot use it another place.

Examples :

```
new int[] {1,2,3,4,5} ;  
new char[] {'x', 'y', 'z'};  
new String[] {"Geeks", "for", "Geeks"};  
new int[][] { {10, 20}, {30, 40, 50} };
```

In the provided example, the anonymous array `arr` is used to store a set of integers temporarily to calculate their sum. This demonstrates the use of an anonymous array for instant usage without the need for a named reference variable.

How many ways to declare array in java?

There are main two ways :

```
1 Using new keyword – String [] names = new String[3];  
names[0] = "Hello";
```

```
names[1] = "Good";
names[2] = "Morning";
```

2 Initialize in One Line – `String [] names = { "Hello", "Good", "Morning"};`

We can use different techniques also like as-

1 Using array Literal – if the size of the array and the variables of the array are already known, array literals can be used. example- `int[] intArray = new int[]{1,2,3,4,5,6,7,8,9,10};`

2 Shorthand Initialization – Java allow it for combining declaration and initialization.

```
int[] myArray = {0,1,2};
```

3 After the Declaration of the array : It is also possible to initialize the array after declaration using new keyword. example – `int [] myarray;`

```
myarray = new int[20];
```

4 Initializing and assigning values together – `int[] num = new int[]{1, 2, 3, 4, 5, 6, 7, 8};`

5 Using array initializer in Constructor : When initializing an array in a constructor, it is important to use the correct syntax, such as `data = new int[] {10, 20, 30, 40};`

What is Jagged Array?

Jagged Array is a concept where we can create matrix every row has different column list.

A jagged array in Java is a multidimensional array where each array can have a different length. It is essentially an array of arrays, allowing for flexibility in the size of each sub-array. This type of array is also known as "ragged array" or an "irregular array".

Jagged array offer a practical solution for scenarios where the data structure is not uniform and can vary in size, making them a valuable tool in java programming.

```
datatype variablename[][] = new datatype[rowsize][] ;
```

```
variablename[rowindex] = new datatype[colsize];
```

Example:

```
import java.util.*;
```

```
public class JaggedArray
```

```
{
```

```
    public static void main(String x[])
```

```
    { int a[][]=new int[3][];
```

```
        a[0]=new int[3];
```

```
        a[1]=new int[4];
```

```
        a[2]=new int[2];
```

```
        Scanner xyz = new Scanner(System.in);
```

```
        System.out.println("Enter values in matrix");
```

```

        for(int i=0; i<a.length;i++)
        {
            for(int j=0; j<a[i].length; j++)
            { a[i][j]=xyz.nextInt();
              }
            }
        System.out.println("Display matrix value");
        for(int i=0; i<a.length;i++)
        {
            for(int j=0; j<a[i].length; j++)
            { System.out.printf("%d\t",a[i][j]);
              }
            System.out.printf("\n");
        }
    }
}

```

How to pass array to the function?

In java, passing an array to function for one-dimensional array or multi-dimensional arrays. The caller function simply calls the called function with the array name as the argument. The number of square brackets in the function prototype is equal to the dimensions of the array.

Example:

```

public class ArrayExample {
    // Method to print elements of a 1D array
    static void printArray(int[] arr) {
        for (int num : arr) {
            System.out.print(num + " ");
        }
    }

    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5};
        printArray(numbers); // Calling the method and passing the array as an argument
    }
}

```

In this example, the printArray method takes an array of integers as a parameter and prints its elements. When calling the printArray method, the numbers array is passed as an argument.

Passing Arrays to Methods in Java – When passing an array to a method in java, the address of the array in the memory is passed, which is the reference. Therefore, any changes to this array in the method will affect the array outside the method.

Overall, passing arrays to methods in java allows for the transfer of a large amount of data between methods, and it's fundamental concept in java programming.

How to copy data from one array to another array?

There are different ways to copy one array to another array –

1 Iterating each element of original array and copy one element at a time –

Example:

```
public class GFG {
    public static void main(String[] args)
    {
        int a[] = { 1, 8, 3 };
        int b[] = new int[a.length];

        // Copying elements of a[] to b[]
        for (int i = 0; i < a.length; i++)
            b[i] = a[i];

        // Changing b[] to verify that b[] is different from a[]
        b[0]++;
        System.out.println("Contents of a[] ");
        for (int i = 0; i < a.length; i++)
            System.out.print(a[i] + " ");

        System.out.println("\n\nContents of b[] ");
        for (int i = 0; i < b.length; i++)
            System.out.print(b[i] + " ");
    }
}
```

2 Using Clone() method –

Example:

```
public class GFG {
    public static void main(String[] args)
    {
        int a[] = { 1, 8, 3 };
        int b[] = a.clone() ;

        // Changing b[] to verify that b[] is different from a[]
        b[0]++;
        System.out.println("Contents of a[] ");
        for (int i = 0; i < a.length; i++)
```

```

        System.out.print(a[i] + " ");

        System.out.println("\n\nContents of b[] ");
        for (int i = 0; i < b.length; i++)
            System.out.print(b[i] + " ");
    }
}

```

3 Using arraycopy() method –

Syntax– `public static void arraycopy(Object src, int srcPos, Object dest, int destPos, int len);`

src : source array

srcPos : index from which copying start

dest : destination array

destPos : index from which copied elements are placed in the destination array.

length : length of subarray to be copied

Example:

```

public class GFG {
    public static void main(String[] args)
    {
        int a[] = { 1, 8, 3 };
        int b[] = new int[a.length];
        System.arraycopy(a, 0, b, 0, 3) ;

        // Changing b[] to verify that b[] is different from a[]
        b[0]++;
        System.out.println("Contents of a[] ");
        for (int i = 0; i < a.length; i++)
            System.out.print(a[i] + " ");

        System.out.println("\n\nContents of b[] ");
        for (int i = 0; i < b.length; i++)
            System.out.print(b[i] + " ");
    }
}

```

4 Using copyOf() method of Arrays class –

Syntax – `public static int[] copyOf?(int[] original, int newLength);`

Example:

```

public class GFG {
    public static void main(String[] args)
    {
        int a[] = { 1, 8, 3 };
    }
}

```

```
int b[] = Arrays.copyOf(a,3) ;

// Changing b[] to verify that b[] is different from a[]
b[0]++;
System.out.println("Contents of a[] ");
for (int i = 0; i < a.length; i++)
    System.out.print(a[i] + " ");

System.out.println("\n\nContents of b[] ");
for (int i = 0; i < b.length; i++)
    System.out.print(b[i] + " ");

}

}
```

5 Using copyOfRange() method of Arrays class –

Syntax – `public static int[] copyOfRange?(int[] original, int from, int to) ;`

Example:

```
public class GFG {  
    public static void main(String[] args)  
    {  
        int a[] = { 1, 8, 3, 5, 9, 10 };  
        int b[] = Arrays.copyOfRange(a, 2, 6) ;  
  
        // Changing b[] to verify that b[] is different from a[]  
        b[0]++;  
        System.out.println("Contents of a[] ");  
        for (int i = 0; i < a.length; i++)  
            System.out.print(a[i] + " ");  
  
        System.out.println("\n\nContents of b[] ");  
        for (int i = 0; i < b.length; i++)  
            System.out.print(b[i] + " ");  
    }  
}
```

How to initialize two-dimensional array in java?

```
int a[][],b,c[]; //here a is two d array b is normal variable and
                  //c is single dimension array

int [][] a,b,c,d; //here all variables are two d array

int[] a[], b,c[][]; //here a is two dimensional array
                    //b is single dimension array
                    and c is three dimensional array.
```

Example:

```
import java.util.*;
public class MatrixApp
{ public static void main(String x[])
  { Scanner xyz = new Scanner(System.in);
    int a[][]=new int[3][3];
    System.out.println("Enter values in matrix");
    for(int i=0; i<a.length;i++)
    {
        for(int j=0; j<a[i].length; j++) {
            a[i][j]=xyz.nextInt();
        }
    }
    System.out.println("Display matrix");
    for(int i=0; i<a.length;i++)
    {
        for(int j=0; j<a[i].length; j++)
        {
            System.out.printf("%d\t",a[i][j]);
        }
        System.out.printf("\n");
    }
  }
}
```

CLASS, Encapsulation, Static, Reference, object

What is class and why use class?

A class is a blueprint for creating objects that shares common properties and methods. It is a template used to create objects and define object datatypes and methods.

For example, in real world, a specific mycat is object of cats class. All cats in the world share some characteristics from the same template, such as being feline, having tail, or being coolest of all animals.

Class is a combination of instance variable, class variable, methods, instance initialize, static initializer, constructor and nested classes. or

Class is combination of state and behaviour.

Instance variable: Those variables declared within class but without static keyword called as instance variable.

Class variable: Those variables declared within class with a static keyword called as class variable

Method: if we define function within class called as method

Instance initializer: if we define block within class called as instance initializer.

Static initializer: if we define block with static keyword called as static initializer.

Constructor: if we define function within class with a same name called as constructor.

Example :

```
class ABC{
    private int x; //instance variable
    private static int y; // class variable or static variable
    public void show() //method
    {
    }
    ABC() //constructor
    {
    }
    static //static block
    {
    }
    class B //Nested class
    {
    }
}
```

Classes are used in java for several reasons:

1 Object Creation – Classes are used to create objects that share common properties and methods.

2 Code Organization – They help in organizing code by grouping related data and functions together.

3 Code Reusability – Classes allow for the reuse of code. Once a class is defined, it can be used to create multiple objects.

4 Encapsulation – Classes facilitate encapsulation, which means the data (attributes) and methods (functions) that operate on the data are bundled together within the class, providing better control over the data.

What is object and explain diff between reference and object?

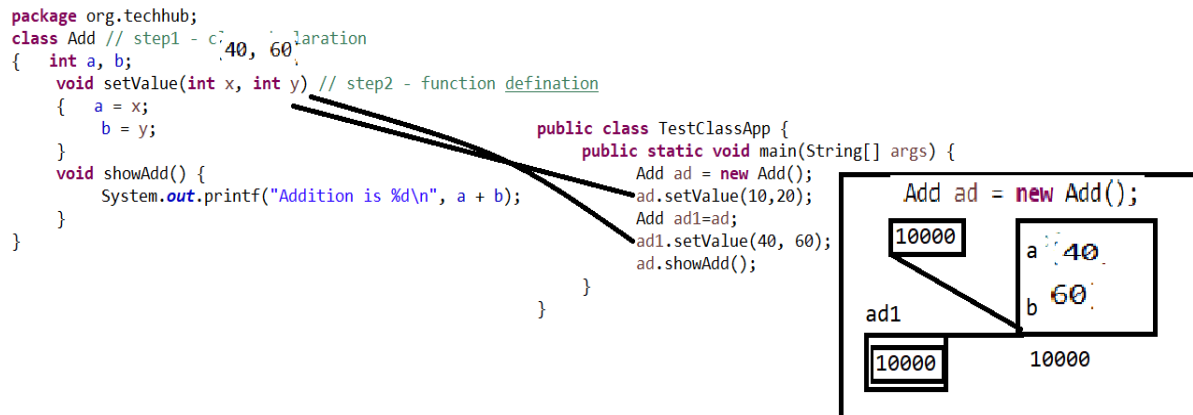
Object is a block of memory where class data store. or

It is instance of class means when we create object of the class then JVM create space for that class in memory and store only non-static member in that.

Syntax : classname ref = new classname() ;

Reference is variable which hold address of object and object is block of memory where class data store.

If we want to use same object multiple time then we can use reference with object.
 When we not use reference with object then JVM create new object every time in memory
 and if we not use reference with object called as anonymous object.
 When we want to use same object more than one time then we can use reference with
 object or store its address in variable.



we can apply multiple reference on single object but if any one reference try to change object content then object previous content may be lost and new content get override on previous content and if we solve this problem java suggest use object cloning concept but will in Object class chapter.

What is method with variable arguments?

Method with variable argument is a facility in java where we can pass infinite parameter to the function.

Syntax:

```
returntype functionname(datatype ...variablename)
{
}
```

... indicate it is infinite array at run time and it is able to pass n number of parameter to the function.

Example:

```
package org.techhub;
class Sum{
    int sum=0;
    void setValue(int ...x) {
        for(int i=0; i<x.length;i++) {
            sum = sum + x[i];
        }
        System.out.printf("Sum of all element is %d\n",s);
    }
}
public class TestVarApp {
```

```

        public static void main(String[] args) {
            Sum s = new Sum();
            s.setValue(10,20,30,40,50);
        }
    }

```

If we think about above code we have class name as Sum with a function void setValue (int ...x) here these ... indicate we pass infinite number of parameter to the function of type integer and we have one more class with a main function and we created object of sum class here Sum s = new Sum() and when we call s.setValue(10,20,30,40,50); here we pass 5 values to sum function of type integer and **it is internally converted as dynamic array**.

What is diff between static and instance variable?

Static Variable	Instance Variable
Declared using static keyword	Declared without static keyword
Exists at the class level and shared among all objects of the class.	Each object of the class has its own copy of the instance variable.
Memory for static variables is allocated only once, at the start of the program execution.	Memory for instance variables is allocated each time an object is created
Can be accessed using the class name- classname.staticvar	Accessed using object reference- obj.instancevar
Example: static int count ;	Example: int age ;
Share its memory common between more than one object of same class.	Share its memory separate for every object in class.

What is this keyword and why use it?

In Java, the "this" keyword refers to the current object in a method or constructor. It is primarily used to eliminate confusion between class attributes and parameters with the same name and has several other uses as well.

If we omit the keyword, it can lead to ambiguity and incorrect output. For example:

```

public class Main {
    int x;
    // Constructor with a parameter
    public Main(int x) {
        this.x = x;
    }
    // Call the constructor
    public static void main(String[] args) {
        Main myObj = new Main(5);
        System.out.println("Value of x = " + myObj.x);
    }
}

```

In this example, using "this" ensures that the correct value is assigned to the class attribute x and the output is "Value of x = 5" .

The "this" keyword can also be used to:

- Invoke the current class constructor
- Invoke the current class method
- Return the current class object
- Pass an argument in the method call
- Pass an argument in the constructor call

The "this" keyword is a crucial tool in Java for referencing the current object and avoiding ambiguity between class attributes and parameters. It is used in various contexts to ensure the correct behavior of methods and constructors within a class

Explain static keyword?

The static keyword in java is mainly used for memory management. It is non access modifier used to share the same variable or method of a given class and belongs to the class rather than instance of the class.

The static keyword can be applied with variables, methods, blocks, and nested classes. When a member of a class is declared as static, it can be accessed without creating an instance of an object for that class.

It is used for a constant variable or a method that is the same for every instance of a class. Static share its single copy among all instances of that class.

```
static String college = "xyz" ;
```

What is static variable ,static method?

Static Variables: When a variable is declared as static, then a single copy of the variable is created and shared among all objects at the class level.

We can create static variables at class level only. i.e. static local variables are not allowed.

```
class Test {  
    public static void main(String args[]) {  
        System.out.println(fun());  
    }  
    static int fun()  
    {  
        static int x= 10; //Error: Static local variables are not allowed  
        return x--;  
    }  
}
```

Static block and static variables are executed in order they are present in a program.

Example:

```

class Test {
    //static variable
    static int a = m1();
    // static block
    static
    {
        System.out.println("Inside static block");
    }
    // static method
    static int m1()
    {
        System.out.println("from m1");
        return 20;
    }
    // static method(main !!)
    public static void main(String[] args)
    {
        System.out.println("Value of a : " + a);
        System.out.println("from main");
    }
}

```

Static method : Any method that uses the static keyword is referred to as a static method.

A static method in java is a part of class rather than an instance of the class.

Every instance of class has access to the method.

Static methods have access to class static variables only without using the class's object(instance).

Only static data may be accessed by a static method. It is unable to access data that is not static (instance variables).

In both static and non static methods, static methods can be accessed directly.

Example:

```

public class Example {
    static int a = 40;
    int b = 50;
    void simpleDisplay() {
        System.out.println(a);
        System.out.println(b);
    }
    static void staticDisplay() {
        System.out.println(a);
    }
    public static void main(String[] args) {
        Example obj = new Example();
        obj.simpleDisplay();
        staticDisplay();
    }
}

```

What is this reference and why use it?

this is a reference variable that refers to the current object.

It used for following purposes:

- Calling current class methods and fields.
- Passing an instance of the current class as a parameter.
- Differentiating between local and instance variables.
- Improving code readability and reducing naming conflicts.

Example:

```
class Animal {
    String name;
    Animal(String name) {
        this.name = name;
    }
    void printDetails() {
        System.out.println("Name: " + this.name);
    }
}

class Dog extends Animal {
    String breed;
    Dog(String name, String breed) {
        super(name); // Calling the constructor of the superclass
        this.breed = breed;
    }
    void printDetails() {
        super.printDetails(); // Calling the method of the superclass
        System.out.println("Breed: " + this.breed);
    }
}

public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog("Buddy", "Labrador");
        myDog.printDetails();
    }
}
```

In above example, this reference is used to differentiate between the fields and methods of the current class and those inherited from the parent class. It allows for accessing and manipulating the current objects instance variables and methods within the context of inheritance.

Can we use this reference with a static variable?

Static variables could be accessed using this but not in the static function. In static block or static method, there is no instance to refer to, and therefore the "this" keyword is not permitted.

But we can refer "this" in a non-static method also we can use refer static variable with "this" keyword in the non-static method. Here, "this" points to current object.

static variable is directly accessible with class name.

Example:

```
public class Example {
    public static int staticVar = 10;

    public static void main(String[] args) {
        // Accessing the static variable using the class name
        int value = Example.staticVar;
        System.out.println("The value of the static variable is: " + value);
    }
}
```

Can we use static variable in non static function?

yes, we can access by using class name followed by the dot operator to reference the static variable. Static variables can be used in any type of methods: static or non-static.

Example:

```
public class Example {
    public static int staticVar = 10;

    public void nonStaticMethod() {
        // Accessing the static variable using the class name
        int value = Example.staticVar;
        System.out.println("The value of the static variable is: " + value);
    }

    public static void main(String[] args) {
        Example obj = new Example();
        obj.nonStaticMethod(); // Calling the non-static method
    }
}
```

Can we use non static variable in static function?

Non static variables cannot be used inside static methods. Because static methods allocate memory before the object creation of program. But non static variables allocate memory after object creation.

What is encapsulation and how we can achieve it Explain with scenario?

Encapsulation means to hide the implementation detail from end user at implementation level called as encapsulation. Means we declare variable of class as private and access via public function called as encapsulation. When we declare variable or data as private then we cannot access that variable outside of class using its object you can access that variable using public function and function contain logic that decide data is accessible or not outside of class called as encapsulation.

In normal terms, Encapsulation is defined as wrapping up data and information under a single unit. Consider a real-life example of encapsulation, in a company, there are different sections like the accounts section, finance section, sales section, etc. Now there may arise a situation when for some reason an official from the finance section needs all the data about sales in a particular month. In this case, he is not allowed to directly access the data of the sales section. He will first have to contact some other officer in the sales section and then request him to give the particular data. This is what encapsulation is. Here the data of the sales section and the employees that can manipulate them are wrapped under a single name "sales section".

Encapsulation also leads to *data abstraction or data hiding*. Using encapsulation also hides the data. In the above example, the data of any of the sections like sales, finance, or accounts are hidden from any other section.

Advantages of encapsulation: Data Hiding, Reusability, Increased flexibility.

Example: Suppose consider we want to design application for stock management of product for small shop and we want to give the two types of logins to application for access database

- 1. for owner login i.e. Admin**
- 2. user login i.e. Employee**

Admin: we want to give all details related product to admin i.e. name, dname, cname, pprice, sprice etc

Employee: we want to give specified info access to employee i.e. name, cname, sprice hide the pprice and dname from employee.

Example:

```
class Person {  
    // Encapsulating the name and age only approachable and used using methods defined  
    private String name;  
    private int age;  
  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```



```

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }
}

public class Main {
    // main function
    public static void main(String[] args)
    {
        // person object created
        Person person = new Person();
        person.setName("John");
        person.setAge(30);

        // Using methods to get the values from the
        // variables
        System.out.println("Name: " + person.getName());
        System.out.println("Age: " + person.getAge());
    }
}

```

What is anonymous object in java?

In java, an anonymous object is an object that is created without giving it a name. Anonymous objects are often used to create objects on the fly and pass them as arguments to methods.

Example:

```

class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

public class Main {

```

```

    public static void main(String[] args) {
        new Person("John", 30).display();
    }
}

```

OUTPUT : Name: John, Age: 30

In this example, we create an anonymous object of the Person class and call its display method. The anonymous object is created using the new keyword and is immediately passed to the display method, without giving it a name. Anonymous objects are often used in combination with inner classes, anonymous classes, and other Java features to perform quick operations without the need to explicitly name the object.

Anonymous objects in Java are useful when a temporary object is needed for a single operation and does not need to persist in memory. They are created on the fly and can be used to perform specific tasks without the overhead of creating a named object.

This example effectively demonstrates the concept of an anonymous object in Java, showcasing the creation and immediate usage of an object without assigning it a name.

How to pass array as parameter in function?

To pass an array as a parameter in a function, we simply pass the array as we would pass any other variable. When an array is passed to a method as an argument, the address of the array in the memory is passed (reference), so any changes to the array in the method will affect the original array.

Example:

```

public class ArrayExample {
    // Method to calculate the sum of elements in an array
    public static int calculateSum(int[] arr) {
        int sum = 0;
        for (int num : arr) {
            sum += num;
        }
        return sum;
    }
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5};
        int sum = calculateSum(numbers);
        System.out.println("Sum of the array elements: " + sum);
    }
}

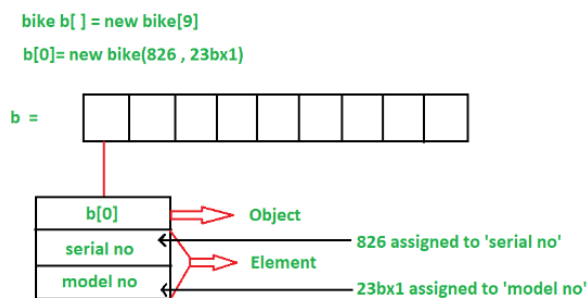
```

What is array of object and how to create it ?

In java, an array of objects is a data structure that stores multiple object references. Unlike traditional arrays that store primitive data types like integers or strings, an array of objects stores references to objects. When creating an array of objects, it's important to note that it's the references to the objects that are stored in the array, not the objects themselves.

An array of Objects is created using the Object class, and we know Object class is the root class of all classes.

Syntax: `ClassName obj[] = new ClassName[ArrayLength] ;`



Different ways to initialize the array of objects:

- 1 By using the constructors.
- 2 By using a separate member method.

1 By using constructor: At the time of creating actual objects, we can assign initial values to each of the objects by passing values to the constructor separately. Individual actual objects are created with their distinct values.

Example:

// Creating a student class with id and name as a attributes

```
class Student {
    public int id;
    public String name;

    Student(int id, String name){    // Student class constructor
        this.id = id;
        this.name = name;
    }

    // display() method to display the student data
    public void display(){
        System.out.println("Student id is: " + id + " " + "and Student name is: "+name);
        System.out.println();
    }
}

class GFG {
```

```

public static void main(String args[]){
    Student[] arr;    // Declaring an array of student

    // Allocating memory for 2 objects of type student
    arr = new Student[2];

    // Initializing the first element of the array
    arr[0] = new Student(1701289270, "Satyabrata");

    // Initializing the second element of the array
    arr[1] = new Student(1701289219, "Omm Prasad");

    // Displaying the student data
    System.out.println("Student data in student arr 0: ");
    arr[0].display();

    System.out.println("Student data in student arr 1: ");
    arr[1].display();
}
}

```

2 By using separate member method: we can initialize objects. A member function of the respective class is created and that is used to assign the initial values to the objects.

Example:

// Creating a Student class with id and name as attributes

```

class Student {
    public int id;
    public String name;

    // Method to set the data to student objects
    public void setData(int id, String name){
        this.id = id;
        this.name = name;
    }

    // display() method to display the student data
    public void display(){
        System.out.println("Student id is: " + id + " " + "and Student name is: "+name);
        System.out.println();
    }
}

class GFG {
    public static void main(String args[]){

        Student[] arr;    // Declaring an array of student

        // Allocating memory for 2 objects of type student

```

```

arr = new Student[2];

// Creating actual student objects
arr[0] = new Student();
arr[1] = new Student();

// Assigning data to student objects
arr[0].setData(1701289270, "Satyabrata");
arr[1].setData(1701289219, "Omm Prasad");

// Displaying the student data
System.out.println("Student data in student arr 0: ");
arr[0].display();

System.out.println("Student data in student arr 1: ");
arr[1].display();
    }
}

```

Where is reference stored in Java and where is Object stored in java?

Classes loaded by ClassLoader and **static variables** and **static object references** are stored in a special location in heap which permanent generation and **Local Variables , references** in stack.

Runtime data area in JVM can be divided as below,

1. **Method Area** : Storage area for compiled class files. (One per JVM instance)
2. **Heap** : Storage area for Objects. (One per JVM instance)
3. **Java stack**: Storage area for local variables, results of intermediate operations. (One per thread)
4. **PC Register** : Stores the address of the next instruction to be executed if the next instruction is native method then the value in pc register will be undefined. (One per thread)
5. **Native method stacks** : Helps in executing native methods (methods written in languages other than Java). (One per thread)

Object and object references are different things.

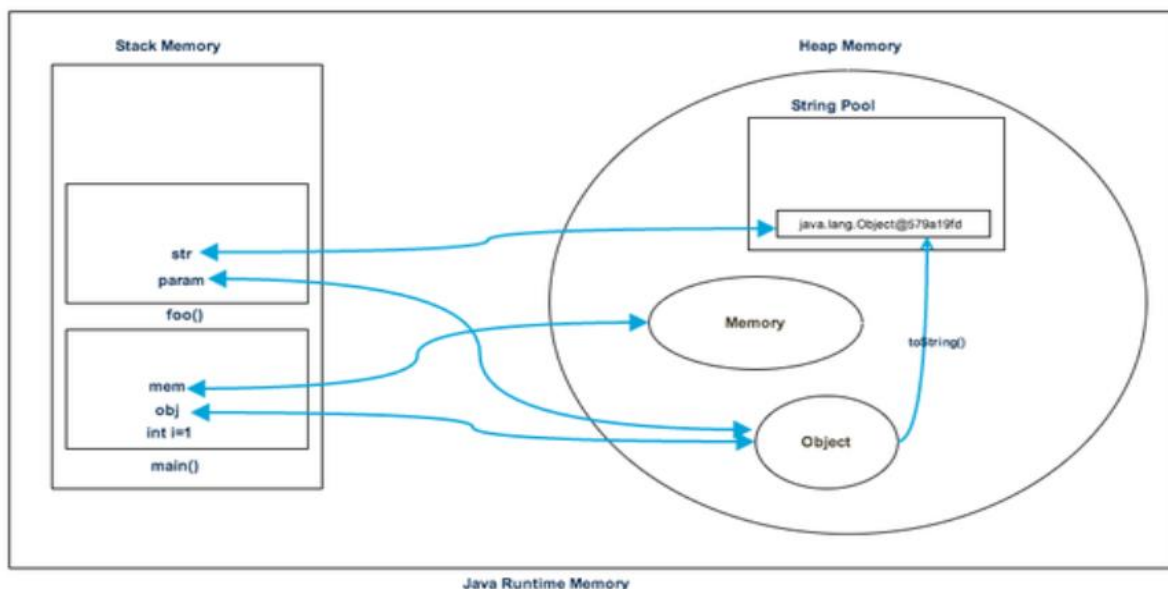
1. There is new keyword in Java used very often to create a new object. But what new does is allocate memory for the object of class you are making and returns a reference. That means, whenever you create an object as static or local, it gets stored in **heap**.
2. All the **class variable primitive or object references** (which is just a pointer to location where object is stored i.e. heap) are also stored in heap.
3. **Classes loaded by ClassLoader** and **static variables** and **static object references** are stored in a special location in heap which permanent generation.

4. **Local primitive variables, local object references** and **method parameters** are stored in **stack**.
5. Local Functions (**methods**) are stored in stack but static functions (methods) goes in permanent storage.
6. **All the information related to a class** like name of the class, object arrays associated with the class, internal objects used by JVM (like Java/Lang/Object) and optimization information goes into the Permanent Generation area.
7. To understand **stack, heap, data** you should read about Processes and Process Control Block in Operating Systems.

The concept is quite simple :

1. Instance variables (primitive, Wrapper classes, references, objects (non-static)) - Heap
2. Local Variables , references - stack
3. Other data objects like : Class metadata, JVM code, static variables, static object references, static functions etc which earlier used to be in Permgen Space (till Java 7) are now being moved to Metaspace in JAVA 8.

PS : Metaspace is part of native memory, so no need to worry about OOM:Permgen Exeption now.



Is true static variable stored within in object if yes then why and if no then why?

Static methods (in fact all methods) as well as static variables are stored in the PermGen section of the heap, since they are part of the reflection data (class related data, not instance related). As of Java 8 PermGen has been replaced by MetaSpace and as per [JEP 122](#) it only holds meta-data while static fields are stored in the heap.

`static int i = 1; //the value 1 is stored in the PermGen section`

static Object o = new SomeObject(); //the reference(pointer/memory address) is stored in the PermGen section, the object itself is not.

PermGen Space is also known as Method Area. PermGen Space used to store 3 things

1. Class level data (meta-data)
2. interned strings
3. static variables

Where static variable present in memory?

Static variable stored in MetaSpace in heap.

Is it true static variable stored in meta section?

yes, static variable stored in meta section.

What is Permanent generation section or META section and why use it?

PermGen Memory: This is a special space in java heap which is separated from the main memory where all the static content is stored in this section. Apart from that, this memory also stores the application metadata required by the JVM.

Metadata is a data which is used to describe the data. Here, garbage collection also happens like any other part of the memory. String pool was also part of this memory before Java 7. Method Area is a part of space in the PermGen, and it is used to store the class structure and the code for methods and constructors.

The biggest disadvantage of PermGen is that it contains a limited size which leads to an OutOfMemoryError. The default size of PermGen memory is 64 MB on 32-bit JVM and 82 MB on the 64-bit version. Due to this, JVM had to change the size of this memory by frequently performing Garbage collection which is a costly operation. Java also allows to manually change the size of the PermGen memory.

However, the PermGen space cannot be made to auto increase. So, it is difficult to tune it. And also, the garbage collector is not efficient enough to clean the memory. PermGen has been completely removed in Java 8. In the place of PermGen, a new feature called Meta Space has been introduced.

What is meta space?

PermGen has been completely removed in Java 8. In the place of PermGen, a new feature called Meta Space has been introduced. MetaSpace grows automatically by default. Here,

the garbage collection is automatically triggered when the class metadata usage reaches its maximum metaspace size.

PermGen	MetaSpace
It is removed from java 8.	It is introduced in Java 8.
PermGen always has a fixed maximum size.	Metaspace by default auto increases its size depending on the underlying OS.
Contiguous Java Heap Memory.	Native Memory(provided by underlying OS).
Inefficient garbage collection.	Efficient garbage collection.

Is it true instance variable allocate memory after object?

yes, instance variable are the variables inside the class so after class object creation memory will allocate to instance variables.

What is garbage collection and what is purpose of Garbage collection?

Garbage collection in java is the automated process of deleting code that's no longer needed or used. This automatically frees up memory space and ideally makes coding Java apps easier for developers. It deletes un referenced objects which has not in use further.

The advantages of Garbage Collection in Java are:

- It makes java memory-efficient because the garbage collector removes the unreferenced objects from heap memory.
- It is automatically done by the garbage collector(a part of JVM), so we don't need extra effort.

Objects can be un referenced in three ways:

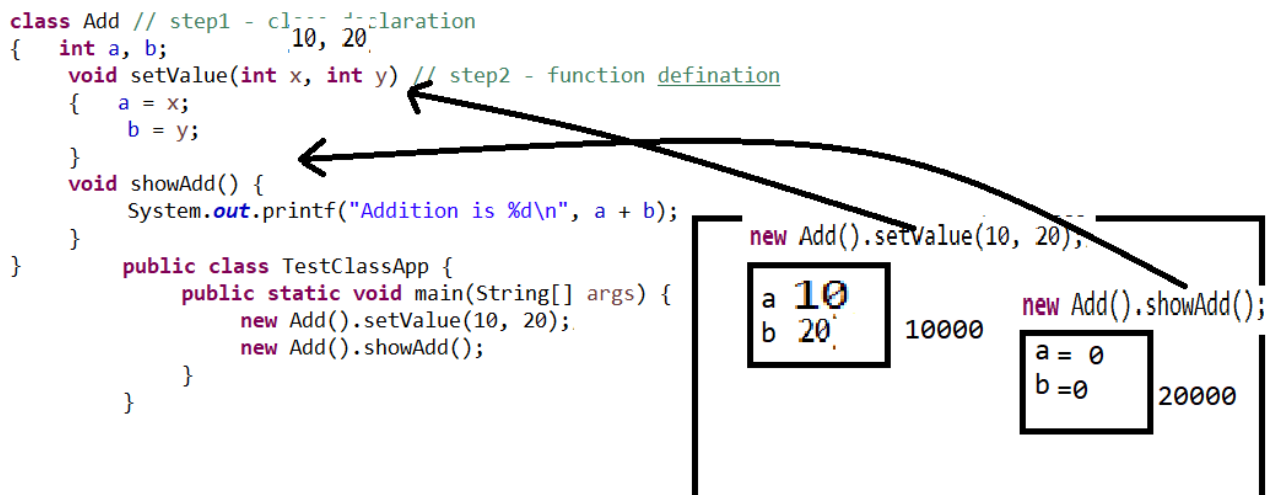
- 1 By nulling the reference
- 2 By assigning a reference to another
- 3 By anonymous object etc.

What happen if we not use reference with object?

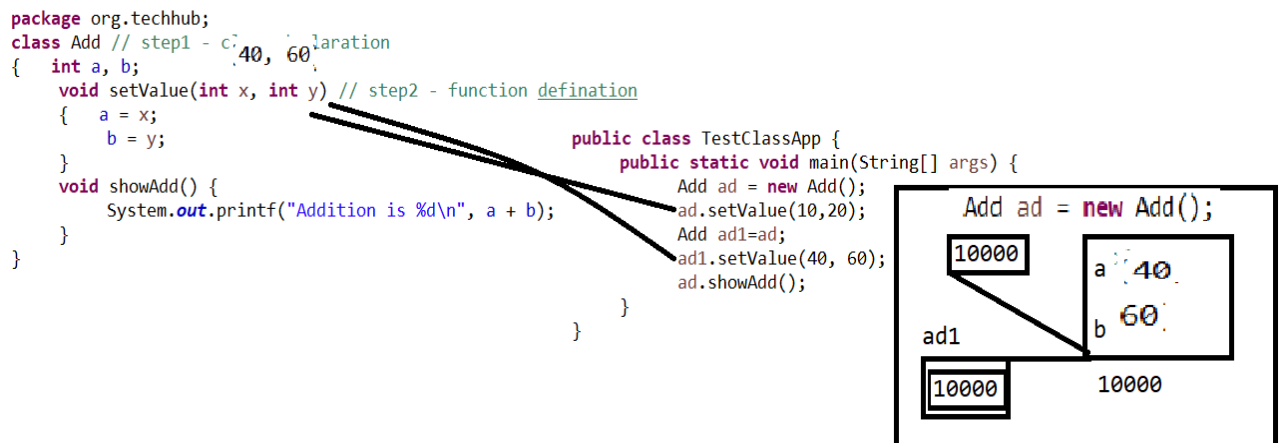
If we want to use same object multiple then we can use reference with object.

When we not use reference with object then JVM create new object every time in memory and if we not use reference with object called as anonymous object.

Following diagram shows what happen if we not use reference with object.



Conclusion: when we use new keyword then JVM create new object every time in memory means new block every time memory and every object has different content.



What is access specifier and explain all types of access specifier in java?

They are the keyword used to apply restrictions on member variables and member methods.

In Java, access specifiers, also known as access modifiers, are keywords used to set the accessibility of classes, interfaces, methods, and other members. There are four types of access specifiers in Java: public, protected, default(package-private), private.

Here's a tabular representation of the visibility of each type of access specifier:

Modifier	Class	Package	Subclass	World
public	Y	Y	Y	Y
protected	Y	Y	Y	N
no modifier	Y	Y	N	N
private	Y	N	N	N

Public: Allows access from anywhere, both within and outside the package.

Protected: Allows access within the same package and by all subclasses of the class, regardless of the package they reside in .

Default (Package-Private): Allows access within the same package, but not by code in subclasses residing in different packages .

Private: Restricts access to only within the same class .

The order of precedence for access specifiers is: public> protected> default(package-private)> private .

Which is default access specifier of java class?

default is a default access specifier of java class.

Is it true we can not declare class as private, static and protected?

No, we cannot declare a top-level class as private or protected. It can be either public or default (no modifier). If it does not have a modifier, it is supposed to have a default access.

- If a top-level class is declared as private the compiler will complain that the modifier private is not allowed here. This means that a top-level class cannot be a private, the same can be applied to protected access specifier also.
- Protected means that the member can be accessed by any class in the same package and by subclasses even if they are in another package.
- The top-level classes can only have public, abstract and final modifiers, and it is also possible to not define any class modifiers at all. This is called default/package accessibility.

- We can declare the inner classes as private or protected, but it is not allowed in outer classes.
- More than one top-level class can be defined in a Java source file, but there can be at most one public top-level class declaration. The file name must match the name of the public class.

Because it makes no sense.

Protected class member (method or variable) is just like package-private (default visibility), except that it also can be accessed from subclasses. Since there's no such concept as 'subpackage' or 'package-inheritance' in Java, declaring class protected or package-private would be the same thing.

You can declare nested and inner classes as protected or private, though.

Is it true public class name and filename should be same?

yes, it is a myth But if we declare class as public then we need to be write same class name and file name, if we not write class as public then we can give different name to file.

```

***** File name: Trial.java *****/
public class Geeks {
    public static void main(String[] args)
    {
        System.out.println("Hello world");
    }
}

```

Output:

```

javac Trial.java
Trial.java:9: error: class Geeks is public, should be
                declared in a file named Geeks.java
public class Geeks
^
1 error

```

```

***** File name: Trial.java *****/
class Geeks {
    public static void main(String[] args)
    {
        System.out.println("Hello world");
    }
}

```

Hello world

what is nested class?

CONSTRUCTOR

What is constructor and why use constructor?

A constructor in Java is a special type of method that is used to initialize objects. It has the same name as the class and does not have a return type. When an object of a class is created, the constructor is automatically called to initialize the object. There are different types of constructors in Java, including the default constructor, parameterized constructor, and copy constructor.

Constructor is function same name as class name without return type.

Need of Constructor:

Think of a Box. If we talk about a box class then it will have some class variables (say length, breadth, and height). But when it comes to creating its object (i.e. Box will now exist in the computer's memory), then can a box be there with no value defined for its dimensions? The answer is **No**.

So constructors are used to assign values to the class variables at the time of object creation, either explicitly done by the programmer or by Java itself (default constructor).

Use of constructor:

- To initialize object

- To call function at time of object creation

- The objects of child class can be initialized by the constructors of base class.

- Encapsulation: Constructors can be used to enforce encapsulation by setting certain instance variables to private and only allowing access to them through public methods.

The constructor(s) of a class must have the same name as the class name in which it resides.

A constructor in Java cannot be abstract, final, static, or Synchronized.

Access modifiers can be used in constructor declaration to control its access i.e. which other class can call the constructor.

What is diff between constructor and function?

- Constructors must have the same name as the class within which it is defined it is not necessary for the method in Java.
- Constructors do not return any type while method(s) have the return type or **void** if does not return any value.
- Constructors are called only once at the time of Object creation while method(s) can be called any number of times.

Constructors	Methods
A Constructor is a block of code that initializes a newly created object.	A Method is a collection of statements which returns a value upon its execution.
A Constructor can be used to initialize an object.	A Method consists of Java code to be executed.
A Constructor is invoked implicitly by the system.	A Method is invoked by the programmer.
A Constructor is invoked when a object is created using the keyword new .	A Method is invoked through method calls.
A Constructor doesn't have a return type.	A Method must have a return type.
A Constructor initializes a object that doesn't exist.	A Method does operations on an already created object.
A Constructor's name must be same as the name of the class.	A Method's name can be anything.
A class can have many Constructors but must not have the same parameters.	A class can have many methods but must not have the same parameters.
A Constructor cannot be inherited by subclasses.	A Method can be inherited by subclasses.

What is constructor chaining and why use it?

Constructor chaining is the process of calling one constructor from another constructor with respect to the current object. It can be done in two ways: Using `this()` constructor , Using `super()` constructor .

means if we call one constructor from another constructor called as constructor chaining.

One of the main uses of constructor chaining is to avoid duplicate codes while having multiple constructors (by means of constructor overloading) and make the code more readable.

It also ensures that the creation of a subclass's object starts with the initialization of the data members of the superclass. This process occurs through inheritance, where a sub-class constructor's task is to call the superclass's constructor first.

Note: Both `this()` & `super()` constructor must be first line of code in calling constructor if we try to write it on different line then compiler will generate compile time error to us , so use one at a time not at same time.

Example:

```

package org.techhub.chaining;
class A {
    A() {
        this(5);
        System.out.println("I am default constructor");
    }
    A(int x) {
        this(5.5f);
        System.out.println("I am integer constructor " + x);
    }
    A(float x) {
        System.out.println("I am float constructor " + x);
    }
}
public class ConsChainApp {
    public static void main(String[] args) {
        A a1 = new A();
    }
}

```

OUTPUT :

```

I am float constructor 5.5
I am integer constructor 5
I am default constructor

```

Explain diff between this () and super () constructor?

1) Using this () constructor: we can perform constructor chaining within single class with constructor overloading

2) Using super () constructor: using super constructor we can perform constructor chaining by using inheritance

Feature	this() Constructor	super() Constructor
Usage	Calls the current class's constructor	Calls the base class's constructor
Syntax	this()	super()
Example	this("parameter")	super("parameter")
Purpose	Used for constructor chaining within the same class	Used for invoking the constructor of the immediate parent class
Context	Typically used for constructor overloading within the same class	Typically used in the context of inheritance to initialize the superclass
Restrictions	Can only be used to call constructors within the same class	Must be the first statement in the constructor of a class

Explain types of constructors?

1) Default constructor: if we do not pass parameter to constructor called as default constructor.

```
class A {  
    A() {System.out.println("I am constructor");  
    }  
}  
  
public class ConsApplication {  
    public static void main(String[] args) {  
  
        A a1 = new A();  
    }  
}
```

Note: if user does not declare constructor in class then compiler add new constructor automatically in class called as implicit constructor.

2) Parameterized constructor: if we pass parameter to constructor called as parameterized constructor when we pass parameter to constructor then we have to pass parameter where we create object of class.

package org.techhub;

```
class Employee{  
    private int id;  
    private String name;  
    public int getId() {  
        return id;  
    }  
    public void setId(int id) {  
        this.id = id;  
    }  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public int getSal() {  
        return sal;  
    }  
    public void setSal(int sal) {  
        this.sal = sal;  
    }  
    private int sal;  
}  
  
class Company
```

```

{ Employee employee;
    Company(Employee employee){
        this.employee=employee;
    }
    void show() {
        System.out.println(employee.getId()+"\t"+employee.getName()+"\t"+employee.getSal());
    }
}

public class CompanyClassApplication {
    public static void main(String[] args) {
        Employee emp = new Employee();
        emp.setId(1);
        emp.setName("ABC");
        emp.setSal(1000);
        Company c = new Company(emp);

//or we can write as
//    Employee emp = new Employee("ABC",1,1000);
//    Company c = new Company(emp);
//or we can write as
//    new Company(new Employee("ABC",1,1000)).show();

        c.show();
    }
}

```

if we think about above code we create object of Employee class and we store data in employee object by using a setter methods but if we want to initialize data in employee object at the time of object creation then you can declare parameterized constructor in Employee class and pass three parameter in it and pass there parameter from Employee object to employee constructor so we not need to call externally setter methods for storing data in object shown in following code.

3 Copy constructor:

Unlike other constructors copy constructor is passed with another object which copies the data available from the passed object to the newly created object.

Note: In Java, there is no such inbuilt copy constructor available like in other programming languages such as C++, instead we can create our own copy constructor by passing the object of the same class to the other instance(object) of the class.

Example:

```

import java.io.*;
class Geek {
    // data members of the class.

```



```

String name;
int id;

// Parameterized Constructor
Geek(String name, int id) {
    this.name = name;
    this.id = id;
}
// Copy Constructor
Geek(Geek obj2){
    this.name = obj2.name;
    this.id = obj2.id;
}
}
class GFG {
    public static void main(String[] args){
        // This would invoke the parameterized constructor.
        System.out.println("First Object");
        Geek geek1 = new Geek("avinash", 68);
        System.out.println("GeekName : " + geek1.name + " and GeekId : " + geek1.id);

        System.out.println();

        // This would invoke the copy constructor.
        Geek geek2 = new Geek(geek1);
        System.out.println("Copy Constructor used Second Object");
        System.out.println("GeekName : " + geek2.name + " and GeekId : " + geek2.id);
    }
}

```

Explain constructor overloading and why use it?

Overloaded constructor means if we use the same name constructor with different type with different parameter list with different parameter sequence called as overloaded constructor.

In the case of constructor overloading which constructor get executed is dependent on its data type, parameter list and parameter sequence.

In Java, constructor overloading serves several important purposes that contribute to code flexibility, maintainability, and ease of use.

Constructor overloading allows for multiple initialization options. Different scenarios may require different sets of initial values for objects, and constructor overloading enables providing multiple ways to initialize objects based on the needs of the user or the context in which the object is created. This leads to a more versatile and adaptable code.

Function call using anonymous object with parameterized constructor using overloading:

```

package org.techhub;
class Cube {

```

```

    int no;
    float value;
    Cube(int no) {
        this.no = no;
    }
    Cube(float value) {
        this.value = value;
    }
    int getIntCube() {
        return no * no * no;
    }
    float getFloatCube() {
        return value * value * value;
    }
}

public class CubeOverloading {
    public static void main(String x[]) {
        int intCube = new Cube(5).getIntCube();
        float floatCube = new Cube(5.5f).getFloatCube();
        System.out.println("Cube of integer is "+intCube);
        System.out.println("Cube of float is "+floatCube);
    }
}

```

Can we declare constructor as private and why use it?

yes, In Java, we can declare a constructor as private using the *private* access specifier. If a constructor is declared private, we can't create an object of the class, except within the class.

A private constructor is used when we want to limit the way objects of a class are instantiated.

Private Constructors can be defined in the Java for the following reasons:

1. To have control on the instantiation of the Java objects, it won't allow you to create an instance of an object.
2. It won't allow the class to be Subclassed.
3. This has a special advantage when implementing the **singleton Pattern**, Private constructors are used for it and have a control on the creating the instance for the whole application.
4. when you want to have a class with all constants defined and does not require its instance anymore, then we declare that class as a private constructor.

What is Singleton class and why use it and explain its code?

Singleton class means class cannot create its more than one object in single application called as Singleton class. Basically Singleton is a design pattern.

A java singleton class is a class that can have only one object (an instance of the class) at a time.

After the first time, if we try to instantiate the Java Singleton classes, the new variable also points to the first instance created. So whatever modifications we do to any variable inside the class through any instance, affects the variable of the single instance created and is visible if we access that variable through any variable of that class type defined.

Remember the key points while defining a class as a singleton class that is while designing a singleton class:

1. Make a constructor private.
2. Write a static method that has the return type object of this singleton class.
Here, the concept of Lazy initialization is used to write this static method.

Purpose of Singleton Class

The primary purpose of a java Singleton class is to restrict the limit of the number of object creations to only one. This often ensures that there is access control to resources, for example, socket or database connection.

Memory space wastage does not occur with the use of the singleton class because it restricts instance creation. As the object creation will take place only once instead of creating it each time a new request is made.

We can use this single object repeatedly as per the requirements. This is the reason why multi-threaded and database applications mostly make use of the Singleton pattern in Java for caching, logging, thread pooling, configuration settings, and much more.

For example, there is a license with us, and we have only one database connection or suppose our JDBC driver does not allow us to do multithreading, then the Singleton class comes into the picture and makes sure that at a time, only a single connection or a single thread can access the connection.

Example:

```
package org.techhub;
class ABC{
    private static ABC ab;
    private ABC() {
        System.out.println("I am constructor");
    }
    public static ABC getInstance() {
        if(ab==null) {
            ab = new ABC();
        }
        return ab;
    }
}
```

```

    }
}
public class SingleTonApplication {
    public static void main(String[] args) {
        ABC a1 = ABC.getInstance();
        ABC a2 = ABC.getInstance();
        ABC a3 = ABC.getInstance();
    }
}

```

What is Utility class in java and how to create it?

A utility class means class contain private constructor and all methods must be static called as utility class.

Example:

```

package org.techhub;
class ABCD{
    private ABCD() {
    }
    public static void test() {
        System.out.println("I am test method from utility class");
    }
    public static void show() {
        System.out.println("I am show method from utility class");
    }
}
public class UtilityClassApplication {
    public static void main(String[] args) {
        ABCD.test();
        ABCD.show();
    }
}

```

Can we declare constructor as static and if yes then why and if no then why?

No, Because static member allocate memory before creating object of class but constructor must be call when we create object of class and it is opposite behavior of each so we cannot declare constructor as static.

Can we perform recursion using a constructor?

No, we can not perform recursion using a constructor. because constructor call when we create object of class but recursion say we need to perform calling within same definition means for recursion we required stack calling but for constructor we required object for calling so this major reason constructor not support to recursion.

Can we provide return type to constructor and what happen if we provide it?

No, Constructor can not have return type. If you try to give return type to constructor then constructor consider as method means in short we can say in java method name and constructor name may be same. so, we need to call it using object name.

EXCEPTIONAL HANDLING

What is exception and why use it?

Exception is an event which occur at program run time and which is responsible for disturb normal flow of application called as exception.

```
import java.util.*;
public class TestDivApp
{
    public static void main(String x[])
    {
        Scanner xyz = new Scanner(System.in);
        int a,b,c;
        System.out.println("Enter two values");
        a=xyz.nextInt();
        b=xyz.nextInt();
        c=a/b;
        System.out.printf("Division is %d\n",c);
    }
}
```



if we think about above code we get run time error java.lang.ArithmeticException because we input two values first is 9 and second is 0 so 9/0 is infinity and system cannot calculate infinity .

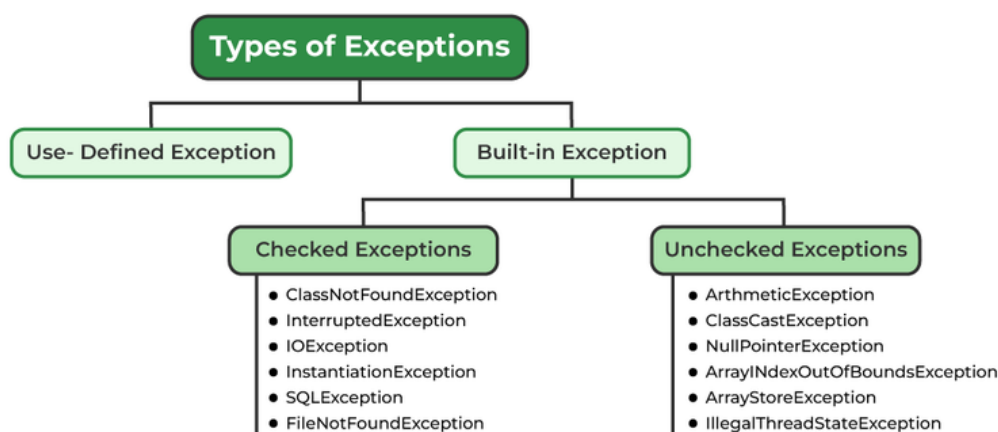
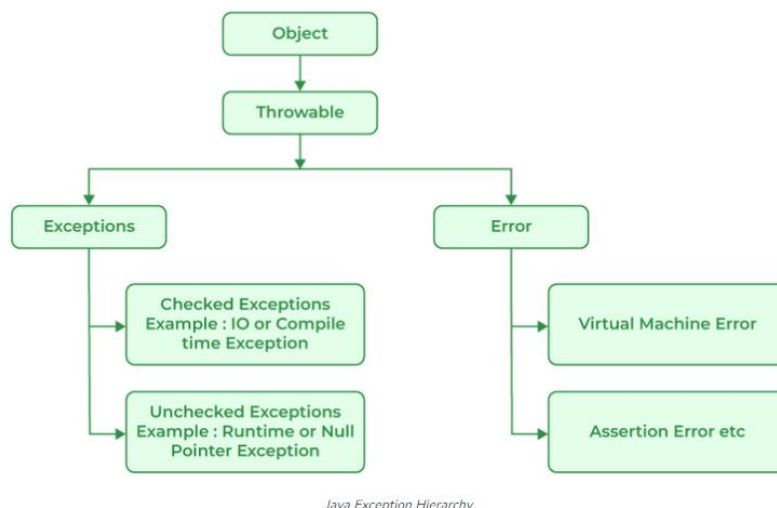
Why use Exception Handling or what is benefit of exception handling?

- 1) Exception Handling help programmer to detect run time error
- 2) Exception Handling skip the code in which exception may be occur and execute remaining program in safe zone.
- 3) User can design own customize in program as per his requirement.

Exception handling in Java is crucial for maintaining the normal flow of a program even when unexpected events occur. It helps in preventing programs from crashing or requests from failing, which can be frustrating for users and lead to customer dissatisfaction.

Reasons of Exception Occurrence: Exceptions in Java can occur due to various reasons, including invalid user input, device failure, loss of network connection, physical limitations (such as out-of-disk memory), and code errors.

**Explains exception handling diagram or hierarchy. or
Explains types of exceptions in java?**



There are three types of exception in Java

1) Checked Exception: those exception occur at program compile time called as checked exception the major goal of checked exception is it provide precaution from exception at run time checked exception help us to generate exception warning at compile time.

They are used to represent problems that can be recovered during the execution of the program. They usually are not the programmer's fault. For example, a file specified by user is not readable, or no network connection available, etc., In all these cases, our program doesn't need to exit, instead it can take actions like alerting the user, or go into a fallback mechanism(like offline working when network not available), etc.

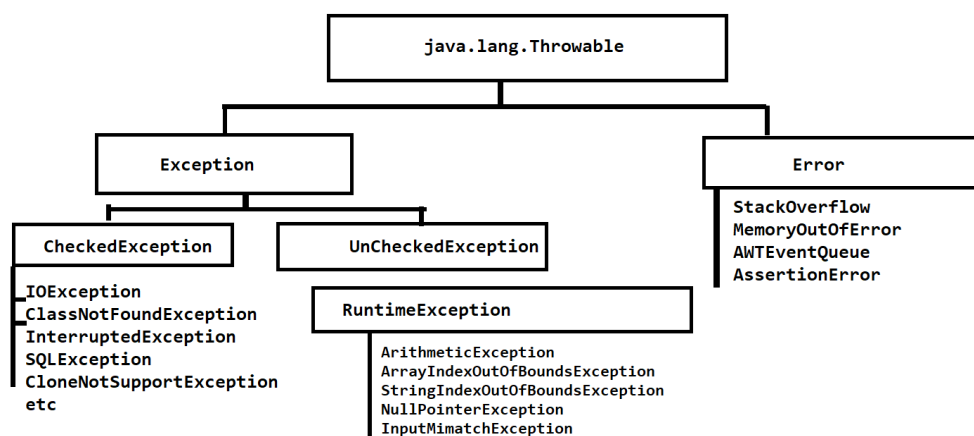
2) Unchecked Exception: unchecked exception means those exception occur at program run time called as unchecked exception.

3) Error: Error is part of exception but the major diff between error and exception is error cannot handle by programmer and exception can handle by programmer.

Means error are system generated

if we want to work with exception handling in java we have some inbuilt classes provided by java to us as well as provide some keywords to us for handle exceptions.

Java Provide some inbuilt keyword to us for handle exception.



try : try is a block which is used for write a code in which exception may be occur when exception generate in program then JVM create exception object in try block as per the exception and pass to catch block means in short we can say we can write code try block there is possibility of exception.

catch: catch block is used for writing logic those want to execute after exception means we can catch block always executed if exception generate by try block means catch block is responsible for handle exception.

Syntax:

```
try{
    write here logics in which exception may be occur
}
catch(exceptiontype ref)
{ write here logics after exception
}
```

single try can have more than one catch block.

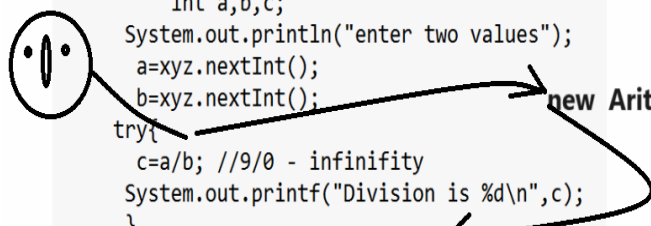
```
try{
```

write here logics in which exception may be occur

```
}  
catch(exceptiontype ref)  
{ write here logics after exception  
}  
catch(exception type ref)  
{  
}
```

finally : finally is block which always execute if exception generate in program or not
we can use finally with try and with try and catch also

```
try{  
  
}  
catch(exception type ref){  
}  
finally{  
}  
or  
try{  
}  
finally{  
}
```



```
import java.util.*;  
public class TestDivApp  
{ public static void main(String x[])  
  {  
    Scanner xyz = new Scanner(System.in);  
    int a,b,c;  
    System.out.println("enter two values");  
    a=xyz.nextInt();  
    b=xyz.nextInt();  
    try{  
      c=a/b; //9/0 - infinity  
      System.out.printf("Division is %d\n",c);  
    }  
    catch(ArithmeticException ex){  
      System.out.println("Error is "+ex);  
    }  
    System.out.println("logic1");  
    System.out.println("logic2");  
    System.out.println("logic3");  
  }  
}
```

Note: if we think about above code, we have statement `c=a/b; //c=9/0` means here internally JVM create object of `ArithmeticException` and hand over to catch block means this object catch by catch block and execute catch and show the error and execute remaining program in safe zone.

Explains all keywords of exception handling with example and why use it?

In Java, exception handling is facilitated using five primary keywords: try, catch, throw, throws, and finally.

1. try: The try block is used to enclose a segment of code that might throw an exception, ensuring that any exception arising from the enclosed code can be gracefully managed.
2. catch: The catch block follows the try block and defines how to handle specific types of exceptions. Multiple catch blocks can be used after a single try to handle different exception types individually.
3. throw: The throw keyword is employed to manually trigger or throw an exception from the code. It's often used in conjunction with user-defined or system exceptions to indicate when something abnormal occurs.
4. throws: The throws keyword is used in the method signature to declare that a method may throw one or more types of checked exceptions. This informs the caller of the method about the potential exceptions that need to be handled.
5. finally: The finally block is used to contain code that must be executed after a try block completes, regardless of whether an exception is thrown or not. It is typically used for cleanup operations, such as closing files or releasing resources.

What is diff between catch and finally block?

Catch block always execute if exception generate in try block but finally execute always.

Catch is responsible handle the exception but finally cannot handle exception.

Catch required parameter of exception type and finally not required parameter.

Can you write try without catch block?

Yes because if we try then using try we can finally directly there is no compulsion to use catch with try every time if we have finally

```
try{
```

```
}
```

```
finally{
```

```
}
```

Example:

```
import java.util.*;
```

```
public class TestDivApp
```

```
{ static int a[];
```

```
public static void main(String x[])
```

```
{
```

```

try{
    String s="12345 ";
    int a =Integer.parseInt(s);
    System.out.println(a);
}
finally{
    System.out.println("I can execute always");
}
}
}
C:\Program Files\Java\jdk1.8.0_291\bin>java TestDivApp
I can execute always
Exception in thread "main" java.lang.NumberFormatException: For input string: "123
45 "
    at java.lang.NumberFormatException.forInputString(NumberFormatException.java:65)
    at java.lang.Integer.parseInt(Integer.java:580)
    at java.lang.Integer.parseInt(Integer.java:615)
    at TestDivApp.main(TestDivApp.java:8)
C:\Program Files\Java\jdk1.8.0_291\bin>

```

If we think about above output here finally block execute first and after that exception occur means finally execute always you can say finally cannot handle exception just execute own block

Note: you can use try, catch and finally at same time then catch handle exception and finally execute its block and you can write finally block after catch.

What is throws clause and why use it?

throws: throws is clause in exception handling which is used for handle the checked exceptions. it only works with a function and when we use throws then we need to write try and catch at function calling point.

Syntax of throws :

```

returntype functionname(arguments)throws exceptiontype
{
}

```

why throws use with a function definition:

we use function for code reusability purpose means when we define any function then we reuse it means function define single time but can use by multiple purpose but when function contain maximum possibility of exception so we need to generate exception warning at function calling pointer or at compile time so this is main reason throws use with function definition.

```

class Div
{
    int a,b;
    void setValue(int x,int y)
    {
        a=x;
        b=y;
    }
    int getDiv()throws Exception{
        return a/b; //10/0
        new ArithmeticException();
    }
}

public class DivApp
{
    public static void main(String x[])
    {
        Div d = new Div();
        d.setValue(10,0);
        try{
            int result=d.getDiv();
            System.out.printf("Division is %d\n",result);
        }
        catch(Exception ex){
            System.out.println("Error Handle At Function Calling "+ex);
        }
    }
}

```

Note: when we use throws clause then exception raise in function definition and handle at function calling point.

```
package org.techhub;
```

```
import java.util.*;
```

```
class Div
```

```
{
    void setValue(int x,int y)throws ArithmeticException,ClassNotFoundException
    {
        System.out.println("Division is "+(x/y));
    }
}
```

```
public class ArrayIndexApplication {
    public static void main(String[] args)
    {
        try {
            Div d = new Div();
            d.setValue(5, 0);
        }
        catch(Exception ex) {
            System.out.println("Error is "+ex);
        }
    }
}
```

What is throw clause and why use it?

throw keyword is used for handle user defined exceptions.

The "throws" clause is essential for ensuring that exceptions are properly handled at the appropriate level of the program. Without it, a method could throw an exception without the calling method being aware of it, potentially leading to unhandled exceptions and unexpected program behavior. By using the "throws" clause, the method effectively communicates to the calling code that it may encounter certain exceptional conditions and requires appropriate handling.

For example, consider a method that performs file I/O operations. If this method encounters an IOException, it can use the "throws" clause to indicate that it may throw this exception.

This alerts the calling code to the possibility of an IOException and ensures that the exception is either caught or propagated to higher levels of the program for handling.

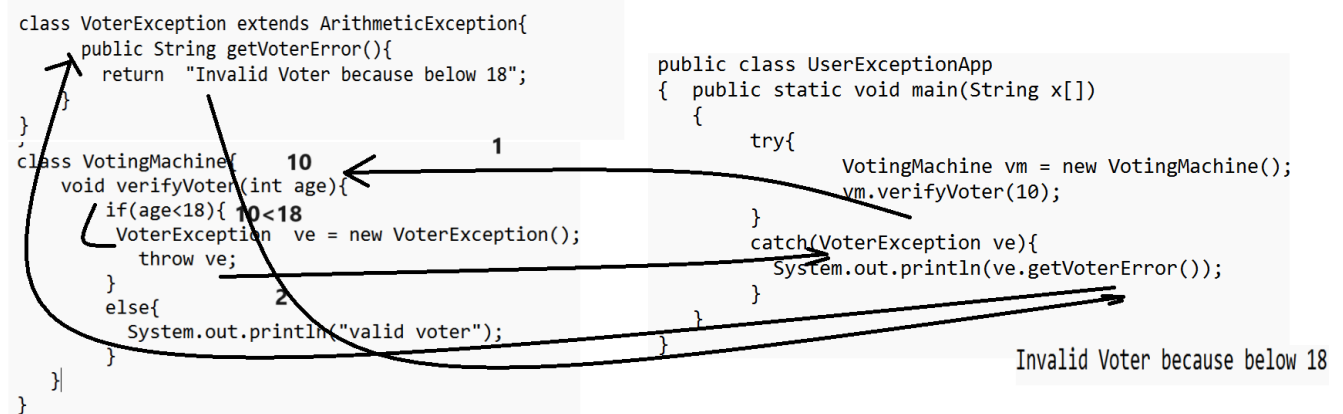
User Defined Exception and why use it?

User defined exception means those exception generated by user or if we user create own classes for exception handling purpose called as user defined exceptions.

There are two reason user can define own exception and handle it.

1. When user has exception and java not provide class for handle that exception then user can write own class for exception handling called as user define exceptions.
2. If user want to handle exception at run time as per his project requirement then user can handle user defined exceptions.

How to handle user define exceptions or How to create user define exceptions



Example:

```
class VoterException extends ArithmeticException{
    public String getVoterError(){
        return "Invalid Voter because below 18";
    }
}

class VotingMachine{
    void verifyVoter(int age){
        if(age<18){
            VoterException ve = new VoterException();
            throw ve;
        }
        else{
            System.out.println("valid voter");
        }
    }
}

public class UserExceptionApp
{ public static void main(String x[])
{
```

```

try{
    VotingMachine vm = new VotingMachine();
    vm.verifyVoter(10);
}
catch(VoterException ve){
    System.out.println(ve.getVoterError());
}
}
}

```

Explains diff between throw and throws?

Throw	Throws
Throw is used for handle user defined exceptions in java	Throws is used for handle checked exception in java
Throw can work with single exception class object at time	Throws can handle or can use multiple exception classes at same time with throws
If use throw, then we need to create manual object exception class and throw it	If we use throws, then we not need to create object of exception class JVM create object of exception class and throw it.

What is diff between exception and error?

Errors	Exceptions
Recovering from Error is not possible.	We can recover from exceptions by either using try-catch block or throwing exceptions back to the caller.
All errors in java are unchecked type.	Exceptions include both checked as well as unchecked type.
Errors are mostly caused by the environment in which program is running.	Program itself is responsible for causing exceptions.
Errors can occur at compile time.	Unchecked exceptions occurs at runtime whereas checked exceptions occurs at comile time
They are defined in java.lang.Error package.	They are defined in java.lang.Exception package
Examples : java.lang.StackOverflowError, java.lang.OutOfMemoryError	Examples : Checked Exceptions : SQLException, IOException Unchecked Exceptions : ArrayIndexOutOfBoundsException, NullPointerException, ArithmeticException.

What is try with resource bundle?

It is a try statement that declares one or more resources, ensuring that each resource is closed at the end of the statement execution. Any object that implements `java.lang.AutoCloseable`, which includes all objects that implement `java.io.Closeable`, can be used as a resource.

Here's an example that demonstrates the use of try-with-resources to read the first line from a file using `FileReader` and `BufferedReader`:

```
try (FileReader fr = new FileReader("file.txt");
    BufferedReader br = new BufferedReader(fr)) {
    String line = br.readLine();
    System.out.println(line);
} catch (IOException e) {
    // Handle exception
}
```

In this example, the `FileReader` and `BufferedReader` are used as resources within the try-with-resources statement. The try-with-resources statement automatically closes these resources at the end of its execution, ensuring proper resource management without the need for explicit finally blocks to close the resources.

Can we write more than one exception in catch block using pipe operator?

Yes, in Java, it is possible to catch multiple exceptions in the same catch block using the pipe (`|`) operator. This feature was introduced in Java 7 and allows a single catch block to handle more than one type of exception.

```
try {
    // Some code that may throw exceptions
} catch (IOException | SQLException e) {
    // Handling code for both IOException and SQLException
}
```

In this example, the catch block is able to handle both `IOException` and `SQLException` using a single catch parameter, separated by the pipe (`|`) operator.

This feature reduces code duplication and increases efficiency, as it allows for more concise and readable exception handling.

Can we write try within try block?

In Java, using a try block inside another try block is permitted. It is called as nested try block.

Example:

```
class Nesting {
    // main method
    public static void main(String args[]) {
```

```

// main try-block
try { // try-block2
    try {
        // try-block3
        try {
            int arr[] = { 1, 2, 3, 4 };
            System.out.println(arr[10]);
        }
        // handles ArithmeticException if any
        catch (ArithmeticException e) {
            System.out.println("Arithmetic exception");
            System.out.println(" try-block1");
        }
    }
    // handles ArithmeticException if any
    catch (ArithmeticException e) {
        System.out.println("Arithmetic exception");
        System.out.println(" try-block2");
    }
}
// handles ArrayIndexOutOfBoundsException if any
catch (ArrayIndexOutOfBoundsException e4) {
    System.out.print("ArrayIndexOutOfBoundsException");
    System.out.println(" main try-block");
}
catch (Exception e5) {
    System.out.print("Exception");
    System.out.println(" handled in main try-block");
}
}
}

```

Can we write multiple catch for single try?

yes,

```

import java.util.Scanner;
public class Test{
    public static void main(String args[]) {
        Scanner scn = new Scanner(System.in);
        try {
            int n = Integer.parseInt(scn.nextLine());

            if (99%n == 0)
                System.out.println(n + " is a factor of 99");
        }
        catch (ArithmeticException ex)
        {
            System.out.println("Arithmetic " + ex);
        }
    }
}

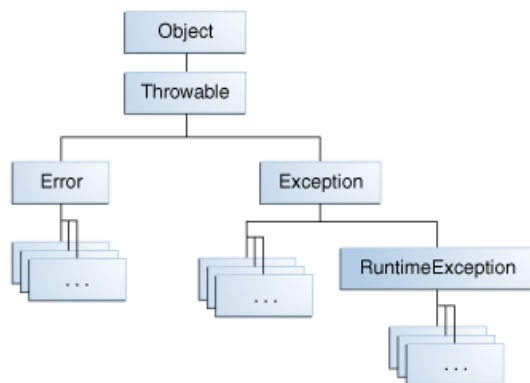
```

```

    }
    catch (NumberFormatException ex)
    {
        System.out.println("Number Format Exception " + ex);
    }
}
}

```

Is it true throw and throws throw exception object at function calling point?



In Java, we can throw anything that extends the Throwable class. However, we don't need to specify a throws clause for all classes. Specifically, classes that are either an Error or RuntimeException or any of the subclasses of these two.

In other case Exception is not a subclass of an Error or RuntimeException. So, it is a checked exception and must be specified in the throws clause, if you don't handle that particular exception. That is why you needed the throws clause.

Is it true throws design for handle checked exception?

yes, A method uses throws clause to specify any exceptions it will raise (or throw). Checked exceptions has to be declared in throws clause otherwise compiler will complain. Unchecked exceptions are optional to be declared in throws clause.

In Java, the 'throws' clause is designed to declare that a method may throw one or more types of checked exceptions. This is a way to indicate to the caller of the method that they need to handle these exceptions. The use of 'throws' is particularly important for checked exceptions.

Explain any 10 exceptions with example?

1. `ArithmeticException`:

- This exception is thrown when an exceptional condition occurs in an arithmetic operation.
- Example:

```
int result = 10 / 0; // This will throw an ArithmeticException
```

2. `ArrayIndexOutOfBoundsException`:

- It is thrown to indicate that an array has been accessed with an illegal index.
- Example:

```
int[] arr = new int[5];  
int value = arr[10]; // This will throw an ArrayIndexOutOfBoundsException
```

3. `ClassNotFoundException`:

- Raised when trying to access a class whose definition is not found.
- Example:

```
try {  
    Class.forName("NonExistentClass");  
} catch (ClassNotFoundException e) {  
    e.printStackTrace();  
}
```

4. `FileNotFoundException`:

- This exception is raised when a file is not accessible or does not open.
- Example:

```
try {  
    FileReader file = new FileReader("NonExistentFile.txt");  
} catch (FileNotFoundException e) {  
    e.printStackTrace();  
}
```

5. `IOException`:

- Thrown when an input-output operation fails or is interrupted.
- Example:

```
try {  
    // Code that performs input-output operations  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

6. `IllegalArgumentException`:

- Represents that an illegal or inappropriate argument has been passed to a method.
- Example:

```
public void setAge(int age) {  
    if (age < 0) {  
        throw new IllegalArgumentException("Age cannot be negative");  
    }  
    this.age = age;  
}
```

7. `NullPointerException`:

- Occurs when a null reference is used where an object is required.
- Example:

```
String str = null;  
int length = str.length(); // This will throw a NullPointerException
```



8. `ClassCastException`:

- Thrown when an invalid cast is made.
- Example:

```
Object obj = new Integer(10);  
String str = (String) obj; // This will throw a ClassCastException
```



9. `DateTimeException`:

- Represents an exception that occurs when manipulating date and time objects.
- Example:

```
LocalDate date = LocalDate.of(2023, 2, 29); // This will throw a DateTimeException
```



10. `ArrayStoreException`:

- Thrown to indicate that an attempt has been made to store the wrong type of object into an array of objects.
- Example:

```
Object[] objArr = new Integer[5];  
objArr[0] = "String"; // This will throw an ArrayStoreException
```



Q10. Why `ArrayIndexOutOfBoundsException` occur?

Q11. Why `NullPointerException` and explain with examples?

Q12. Explain `NumberFormatException` and its reason?

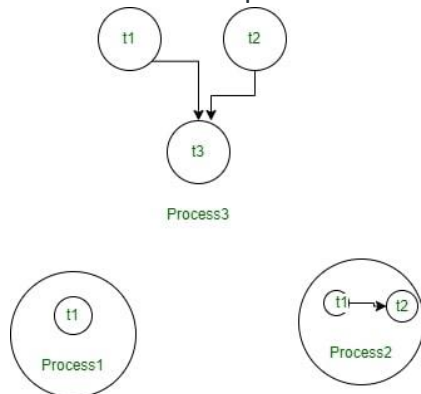
Q13. Explain `StringIndexOutOfBoundsException`, `ArithmeticException`, `ClassNotFoundException` etc

Q14. What is error and explain any 5 error classes in java?

THREADING

What is thread and why use it?

Thread is a sub-part of process. These threads use shared memory but they act independently hence if there is an exception in threads that do not affect the working of other threads despite them sharing the same memory.



As we can observe in, the above diagram a thread runs inside the process and there will be context-based switching between threads there can be multiple processes running in OS, and each process again can have multiple threads running simultaneously. The Multithreading concept is popularly applied in games, animation...etc.

Why use Thread in Java:

the advantage of being lightweight and can provide communication between multiple threads at a Low Cost contributing to effective multi-tasking within a shared memory environment.

Efficient Program Operation: Threads allow a program to operate more efficiently by doing multiple things at the same time. They can be used to perform complicated tasks in the background without interrupting the main program.

Handling Multiple Tasks in Parallel: Multi-threading is used to handle multiple tasks in parallel. By creating multiple Java threads, each performing a different task, a program can effectively handle multiple operations simultaneously.

Dividing Jobs into Independent Parts: Multithreading is used when a job can be divided into several independent parts. For example, in scenarios where complex database operations need to be executed, multithreading can be beneficial.

Enabling Multiple Operations within a Single Method: Threads are critical in a program because they enable multiple operations to take place within a single method. Each thread often has its own program counter, stack, and local variable, allowing for concurrent execution of different parts of the program.

What is multithreading and its benefits?

Multithreading means to execute multiple thread simultaneously in waiting of each other. or to utilize idle period of processor called as multi-threading.

Benefits of multithreading:

1. Improved Throughput: Multithreading allows for many concurrent compute operations and I/O requests within a single process, leading to improved throughput.

2. Utilization of Multiple Processors: Multithreading enables the simultaneous and fully symmetric use of multiple processors for computation and I/O, thereby maximizing the utilization of available hardware resources.

3. Superior Application Responsiveness: By launching requests on their own threads, applications do not freeze or show the "hourglass," leading to superior application responsiveness. This means that an entire application will not block or wait pending the completion of another request.

4. Efficient Resource Sharing: All the threads of a process share its resources such as memory, data, and files, leading to efficient resource sharing and utilization.

5. Maximum CPU Utilization: Multithreading leads to maximum utilization of the CPU by multitasking, allowing for the execution of multiple parts of a program at the same time.

What is process?

Process is a running program in java. A process can create other processes which are known as Child Processes. The process takes more time to terminate and it is isolated means it does not share the memory with any other process.

The process can have the following states new, ready, running, waiting, terminated, and suspended.

What is difference between process and thread?

Process	Thread
Process means any program is in execution.	Thread means a segment of a process.
The process takes more time to terminate.	The thread takes less time to terminate.
It takes more time for creation.	It takes less time for creation.
It also takes more time for context switching.	It takes less time for context switching.
The process is less efficient in terms of communication.	Thread is more efficient in terms of communication.
Multiprogramming holds the concepts of multi-process.	We don't need multi programs in action for multiple threads because a single process consists of multiple threads.
The process is isolated.	Threads share memory.
The process is called the heavyweight process.	A Thread is lightweight as each thread in a process shares code, data, and resources.
Process switching uses an interface in an operating system.	Thread switching does not require calling an operating system and causes an interrupt to the kernel.
If one process is blocked then it will not affect the execution of other processes	If a user-level thread is blocked, then all other user-level threads are blocked.
The process has its own Process Control Block, Stack, and Address Space.	Thread has Parents' PCB, its own Thread Control Block, and Stack and common Address space.
Changes to the parent process do not affect child processes.	Since all threads of the same process share address space and other resources so any changes to the main thread may affect the behavior of the other threads of the process.
A system call is involved in it.	No system call is involved, it is created using APIs.
The process does not share data with each other.	Threads share data with each other.

How many ways to create thread in java explain with example?

There are two ways to implement thread in java:

- 1 Using thread class
- 2 Using Runnable interface

Using thread class –

```
package org.techhub;
class MyThread extends Thread {
    public void run() {
        try {
            for (int i = 1; i <= 5; i++) {
                System.out.println("First Thread  "+i+"\t"+this.isAlive());
                if(i==3) {
                    stop();
                }
                sleep(10000);
            }
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}

class MyThread1 extends Thread{
    public void run() {
        try {
            for (int i = 1; i <=50; i++) {
                System.out.println("Second Thread  "+i);
                sleep(1000);
            }
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}

public class MyThreadApplication {
    public static void main(String[] args) throws InterruptedException{
        MyThread m = new MyThread();
        m.start();
        m.join();
        System.out.println("Now Thread status is "+m.isAlive());
        MyThread1 m1 = new MyThread1();
        m1.start();
    }
}
```

```
}
```

Note: Every program contains by default thread called as main thread and it is start by JVM internally.

Using Runnable Interface:

```
package org.techhub;
class Test implements Runnable
{
    @Override
    public void run() {
        try {
            for(int i=1;i<=5; i++) {
                System.out.println("I = "+i);
                Thread.sleep(1000);
            }
        }
        catch(Exception ex) {
            System.out.println("Error is "+ex);
        }
    }
}

public class ThreadRunnableApp {
    public static void main(String[] args) {
        Test t1 = new Test();
        Thread th = new Thread(t1);
        th.start();
    }
}
```

Write a code for Create thread using Thread class?

```
package org.techhub;
class A extends Thread{
    public void run() {
        try {
            for(int i=1; i<=5; i++) {
                System.out.println("First thread is "+i);
                Thread.sleep(10000);
            }
        }
        catch(Exception ex) {
            System.out.println("Error is "+ex);
        }
    }
}
```

```

}
class B extends Thread{
    public void run() {
        try {
            for(int i=1; i<=50; i++) {
                System.out.println("Second thread is "+i);
                Thread.sleep(1000);
            }
        }catch(Exception ex) {
            System.out.println("Error is "+ex);
        }
    }
}
}
public class ThreadTestApp {
    public static void main(String[] args) {
        A a1 = new A();
        a1.start();//run() call automatically
        B b1 = new B();
        b1.start();
    }
}

```

Explain Thread class all methods?

void start(): this method help us to start thread and when we call start() method then internally run() method get executed.

static void sleep(int milliseconds): this method can hold thread execution for specified time period.

void join():method help us to hold another thread execution when ever running thread is not completed.

boolean isAlive(): this method help us to check thread is working or not if thread is working return true otherwise return false.

void stop(): this method help us stop the thread or terminate the thread.
etc

boolean isAlive(): this method is used for check the status of thread means thread is live or not if thread is live return true otherwise return false.

void stop(): this method help us to stop the thread or terminate the execution of thread.

Is it true Runnable contain run() method?

public void run()	This method takes in no arguments. When the object of a class implementing Runnable class is used to create a thread, then the run method is invoked in the thread which executes separately.
----------------------	---

In this method, we have the code which we want to execute on a concurrent thread. In this method, we can use variables, instantiate classes, and perform an action like the same way

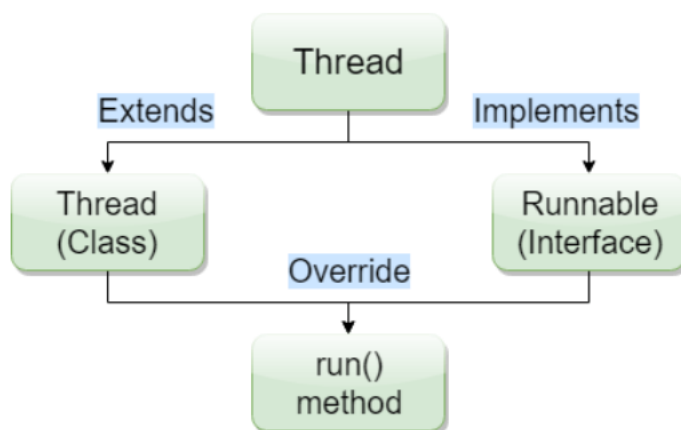
the main thread does. The thread remains until the return of this method. The run method establishes an entry point to a new thread.

```
1. public class ExampleClass implements Runnable {
2.
3.     @Override
4.     public void run() {
5.         System.out.println("Thread has ended");
6.     }
7.
8.     public static void main(String[] args) {
9.         ExampleClass ex = new ExampleClass();
10.        Thread t1 = new Thread(ex);
11.        t1.start();
12.        System.out.println("Hi");
13.    }
14. }
```

OUTPUT:

Hi

Thread has ended



There are several differences between Thread class and Runnable interface based on their performance, memory usage, and composition.

- By extending thread, there is overhead of additional methods, i.e. they consume excess or indirect memory, computation time, or other resources.
- Since in Java, we can only extend one class, and therefore if we extend Thread class, then we will not be able to extend any other class. That is why we should implement Runnable interface to create a thread.

- Runnable makes the code more flexible as, if we are extending a thread, then our code will only be in a thread whereas, in case of runnable, one can pass it in various executor services, or pass it to the single-threaded environment.
- Maintenance of the code is easy if we implement the Runnable interface.

Is it true Runnable interface is implemented by Thread class?

Yes, the Runnable interface is implemented by the Thread class in Java. The `java.lang.Runnable` is an interface that is to be implemented by a class whose instances are intended to be executed by a thread. There are two ways to start a new Thread: subclass Thread and implement Runnable. When a task can be done by overriding only the `run()` method of Runnable, there is no need for subclassing a Thread. The Thread class has a constructor which accepts Runnable instances, and invoking `start()` of the Thread instance creates a new Thread that executes the code written in `run()`.

Is it true Runnable is marker interface?

No, the Runnable interface is not a marker interface. A marker interface is an interface that does not contain any methods, fields, or constants. It serves as a tag to indicate to the JVM and compiler that objects of the implementing class should be treated differently. Examples of marker interfaces in Java include `Serializable` and `Cloneable`. However, the Runnable interface does contain a method to be implemented, namely the `run()` method. Therefore, it does not fit the definition of a marker interface.

Runnable is not a marker a interface as it contains a method.
Interfaces without any methods are marker interfaces. by having the class implement it, you don't impose any functionality on the class.

is it true every java program contains at least one thread?

Threads are sometimes called *lightweight processes*. Both processes and threads provide an execution environment, but creating a new thread requires fewer resources than creating a new process.

Threads exist within a process — every process has at least one. Threads share the process's resources, including memory and open files. This makes for efficient, but potentially problematic, communication.

Multithreaded execution is an essential feature of the Java platform. Every application has at least one thread — or several, if you count "system" threads that do things like memory management and signal handling. But from the application programmer's point of view, you start with just one thread, called

the *main thread*. This thread has the ability to create additional threads, as we'll demonstrate in the next section.

Generally, all the programs have at least one thread, known as the main thread, that is provided by the JVM or Java Virtual Machine at the starting of the program's execution. At this point, when the main thread is provided, the `main()` method is invoked by the main thread. A thread is an execution thread in a program.

Is it true we cannot any java program without thread?

It is indeed possible to run a Java program without explicitly creating additional threads. By default, every Java program starts with a main thread, which is created by the Java Virtual Machine (JVM) to execute the main method. This main thread can handle the program's execution without the need for additional threads.

However, it's important to note that while it's possible to run a Java program without explicitly creating additional threads, the use of multiple threads can provide benefits such as concurrent execution and improved performance in certain scenarios.

Therefore, while it's not a strict requirement to create additional threads in every Java program, the use of threads can offer advantages in handling concurrent tasks and optimizing performance.

In summary, it is possible to run a Java program without explicitly creating additional threads, as the main thread provided by the JVM can handle the program's execution.

is it true run () method is not a member of Thread class?

Q8. is it true Thread class is implementer class of Runnable interface?

Q9. what is thread priority and why use it explain all types of thread priority?

Q10. Explain all types of thread priority in java?

Q11. what is default priority of thread in java?

Q12. what is use of sleep() method of Thread class?

Q13. is it true for sleep () method must be handle InterruptedException?

Q14. what is use of volatile keyword?

Q15. Explain thread life cycle in java?

- Q16. What is used for wait () method and explain types of wait?
- Q18. What daemon threads in java and explain some names of daemon thread in java?
- Q19. Can we create your own thread as daemon thread in java?
- Q20. How to get the priority of thread in java?
- Q21. Can we use wait (), notify () and notifyAll() in non-synchronized block?
- Q22. How to get current running thread in java?
- Q23. what is diff between multi-tasking and multi-threading in java?
- Q24. what is synchronization and asynchronization in thread?
- Q13. Explain isAlive(), join(), start(),
run(),sleep(),stop(),yield(),setPriority(),getPriority(),wait(),notify() and notifyAll() methods?
- Q25. What is use of join () and isAlive () method?
- Q26. What is diff between wait () and sleep () method?
- Q27.what is use of notify () and notifyAll ()
- Q28. What is diff between notify () and notifyAll () method?
- Q29. Why we need to create thread using Runnable interface?
- Q30 Write code for creating thread using Runnable interface?